



深度学习平台与应用

第十一讲：视频理解

■ 分类、分割、检测

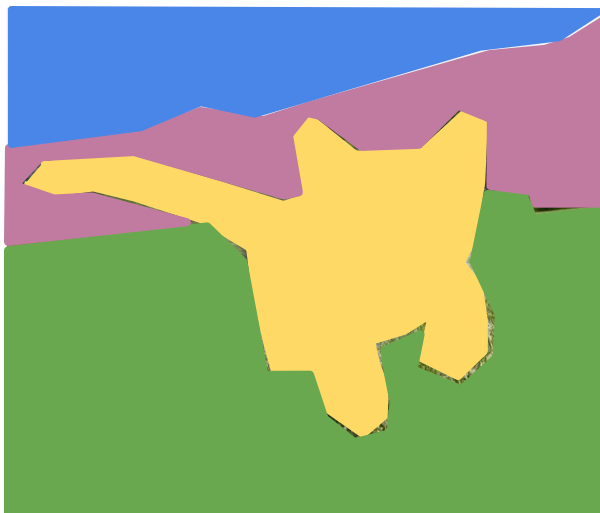
Classification



CAT

No spatial extent

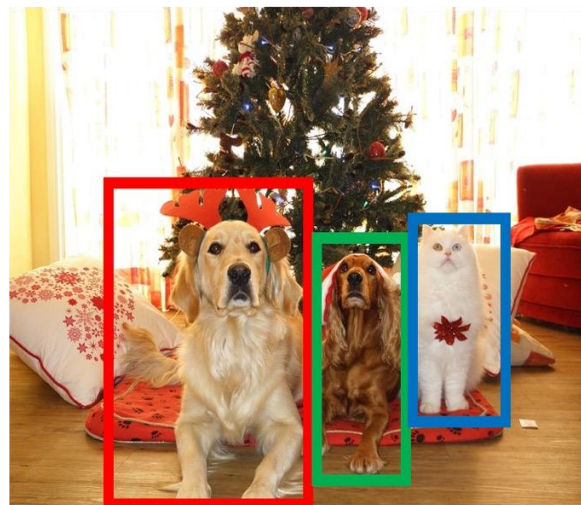
Semantic
Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object
Detection



DOG, DOG, CAT

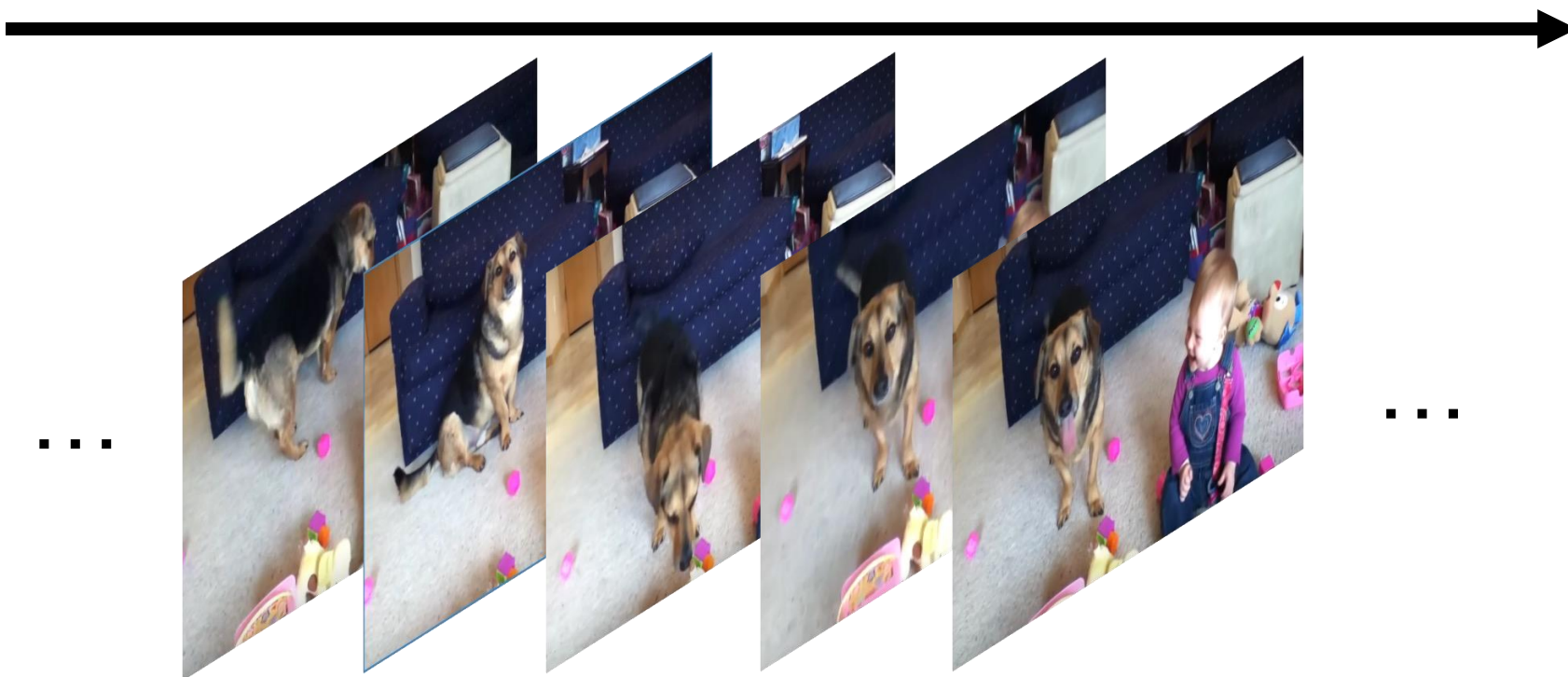
Multiple Object

Instance
Segmentation



DOG, DOG, CAT

- Video = 2D + Time
- 视频由图片序列组成: $T \times 3 \times H \times W$



■ 视频分类



Input video:
 $T \times 3 \times H \times W$



Swimming
Running
Jumping
Eating
Standing

■ 视频分类



图片：识别类别



Dog
Cat
Fish
Truck



视频：识别动作



Swimming
Running
Jumping
Eating
Standing

- **问题：视频太大了！**
- **30 FPS 的视频：**
 - **SD (640 x 480): 每分钟 ~1.5 GB**
 - **HD (1920 x 1080): 每分钟 ~10 GB**



Input video:
 $T \times 3 \times H \times W$

- 问题：视频太大了！
- 30 FPS 的视频：
 - SD (640 x 480): 每分钟 ~1.5 GB
 - HD (1920 x 1080): 每分钟 ~10 GB
- 解决方案：
 - 在切片 (clips) 上训练, 低 FPS, 低分辨率
 - $T=16$, $H=W=112$ 时, 3.2s 视频 588KB 大小



Input video:
 $T \times 3 \times H \times W$

■ 视频切片 (clips) 训练

原始视频：长视频, 高 FPS



■ 视频切片 (clips) 训练

原始视频：长视频, 高 FPS



训练：在低 FPS 的 clips 上训练



■ 视频切片 (clips) 训练

原始视频：长视频, 高 FPS



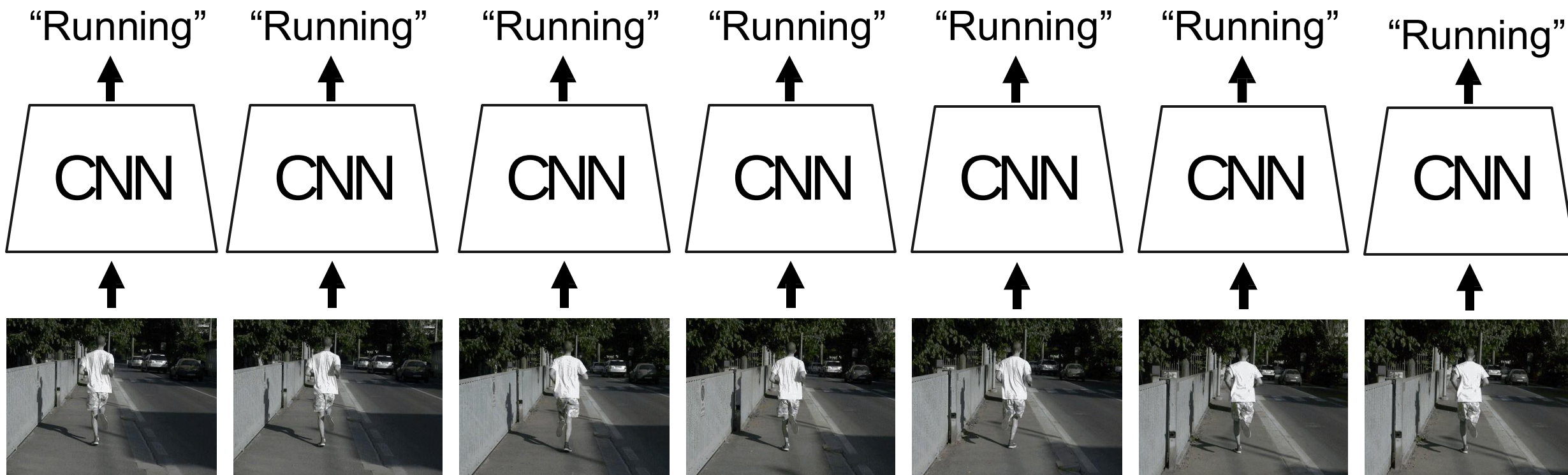
训练：在低 FPS 的 clips 上训练



测试：在不同 clips 上运行模型，取平均预测结果



- 单帧 CNN 模型：训练普通的 2D CNN 独立对视频帧进行分类
- 通常是视频分类的一个非常强的基线方法



■ Late Fusion (FC) : 将每帧的 CNN 特征并联并送入 MLP

问题?

✖

Clip features: $TDH'W$ MLP ➔ Class scores: C



Flatten

Frame features

$T \times D \times H' \times W'$

2D CNN on
each
frame

CNN

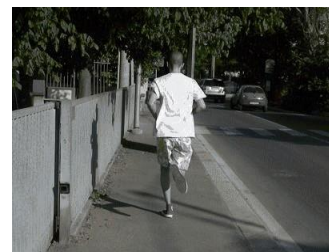
CNN

CNN

CNN

CNN

CNN

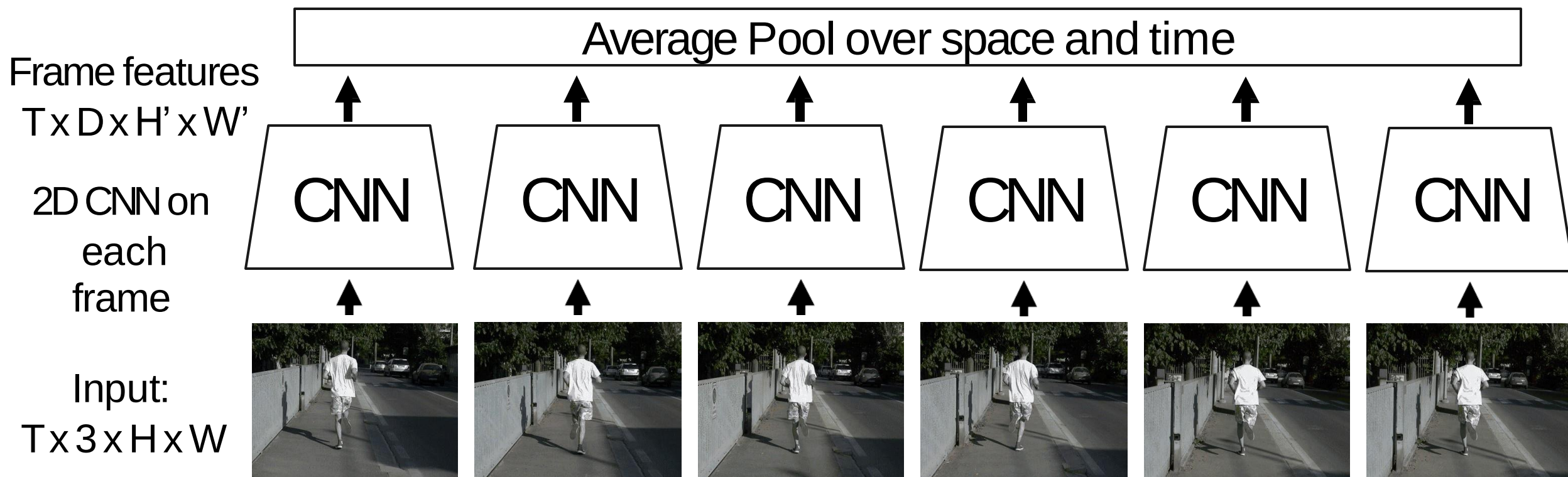


Input:
 $T \times 3 \times H \times W$

■ Late Fusion (Pooling) : 将每帧 CNN 特征池化并送入线性层

问题?

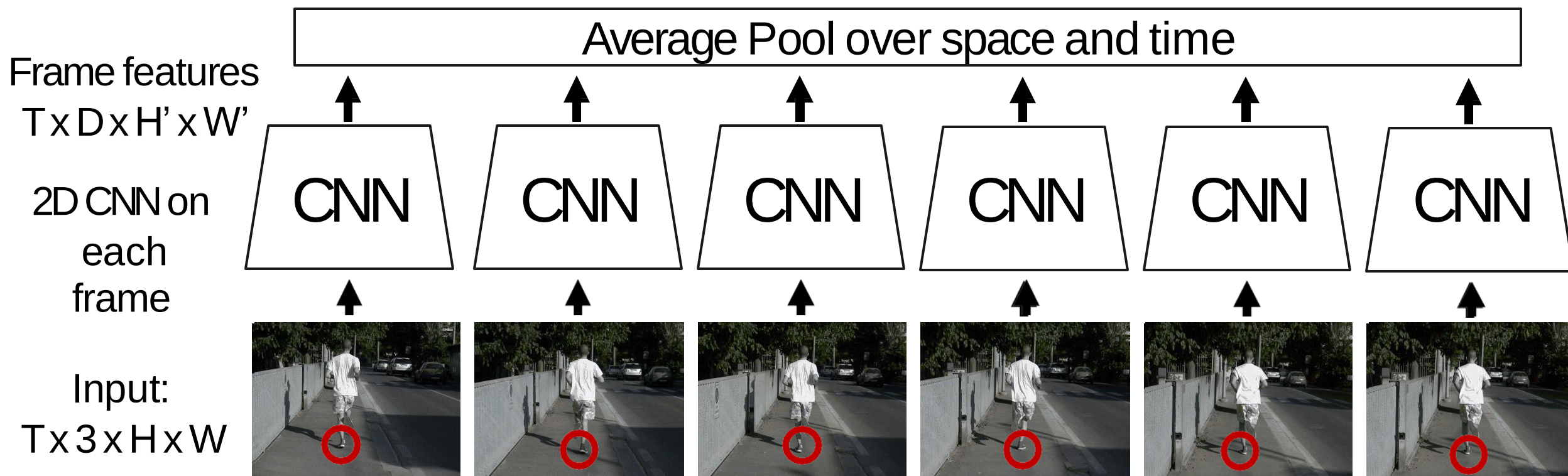
Clip features: $TDH'W$ MLP $\rightarrow$ Class scores: C



■ Late Fusion (Pooling) : 将每帧 CNN 特征池化并送入线性层

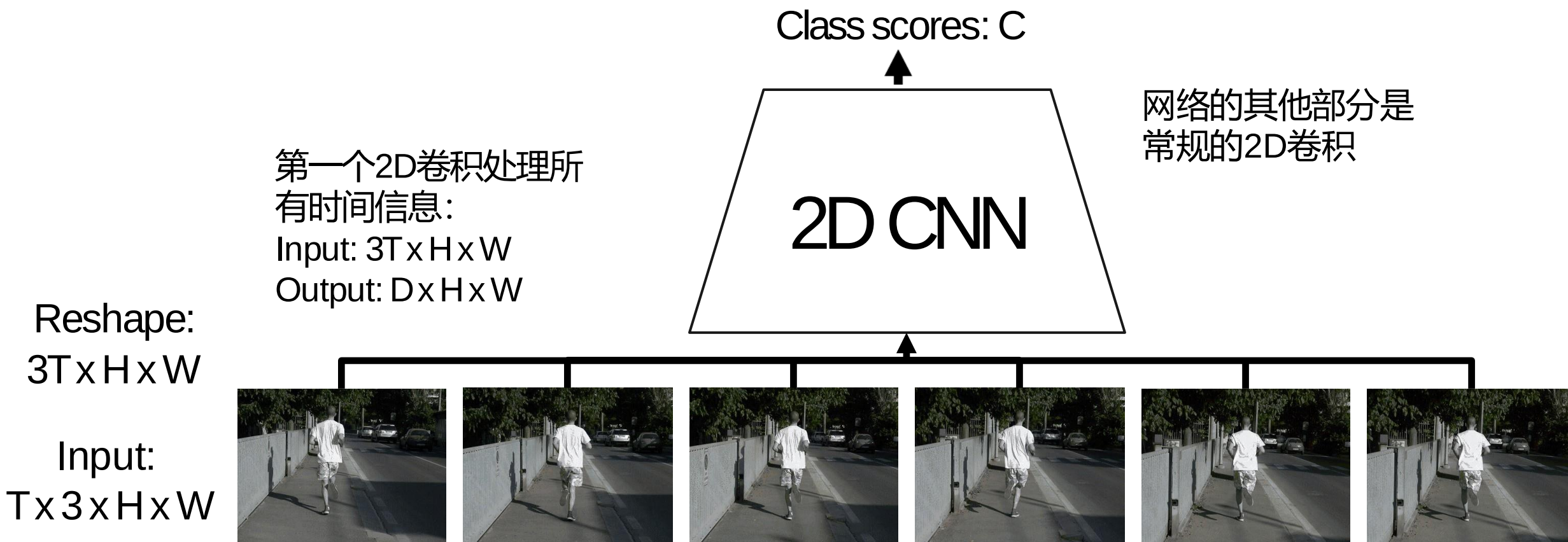
问题: 难以捕捉帧之间的
low-level motion

Clip features: $T \times D \times H' \times W'$ MLP $\rightarrow$ Class scores: C



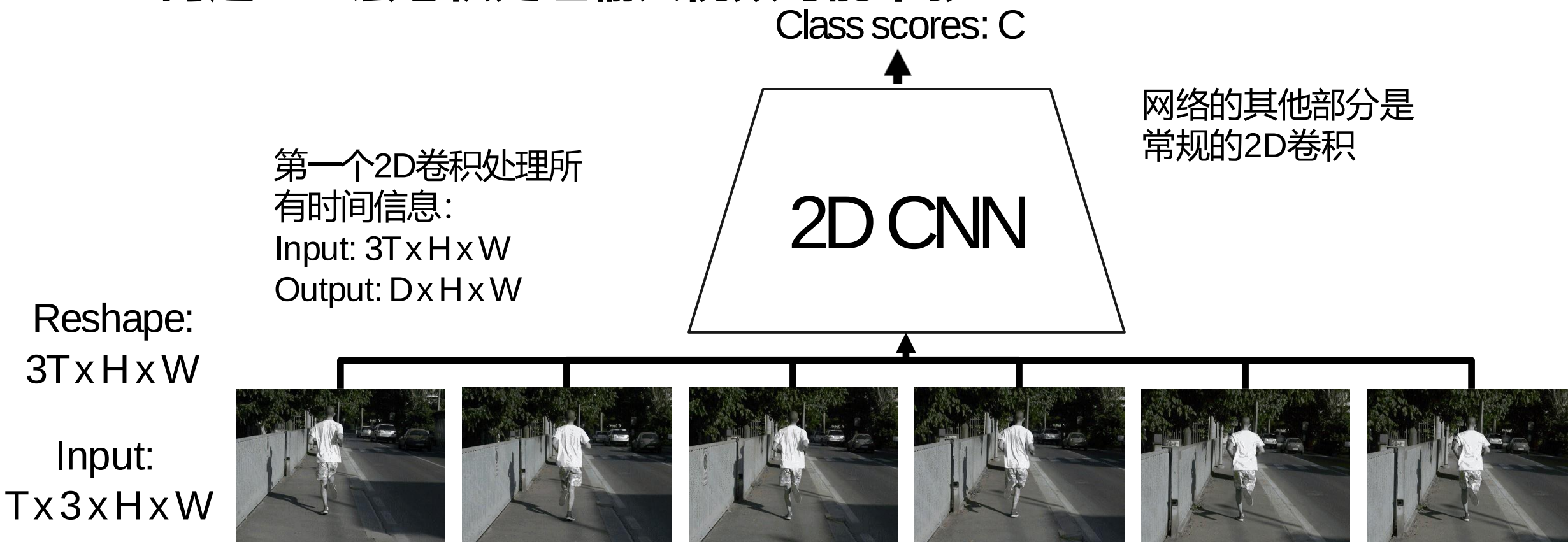
■ Early Fusion

问题?

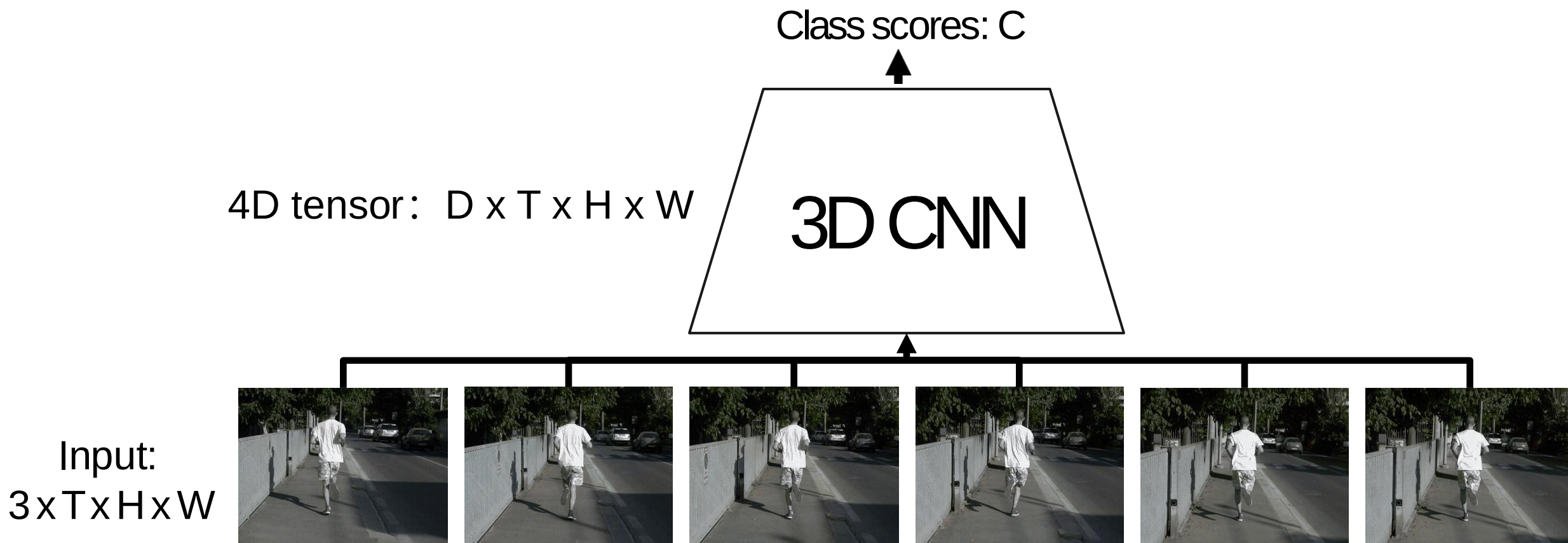


■ Early Fusion

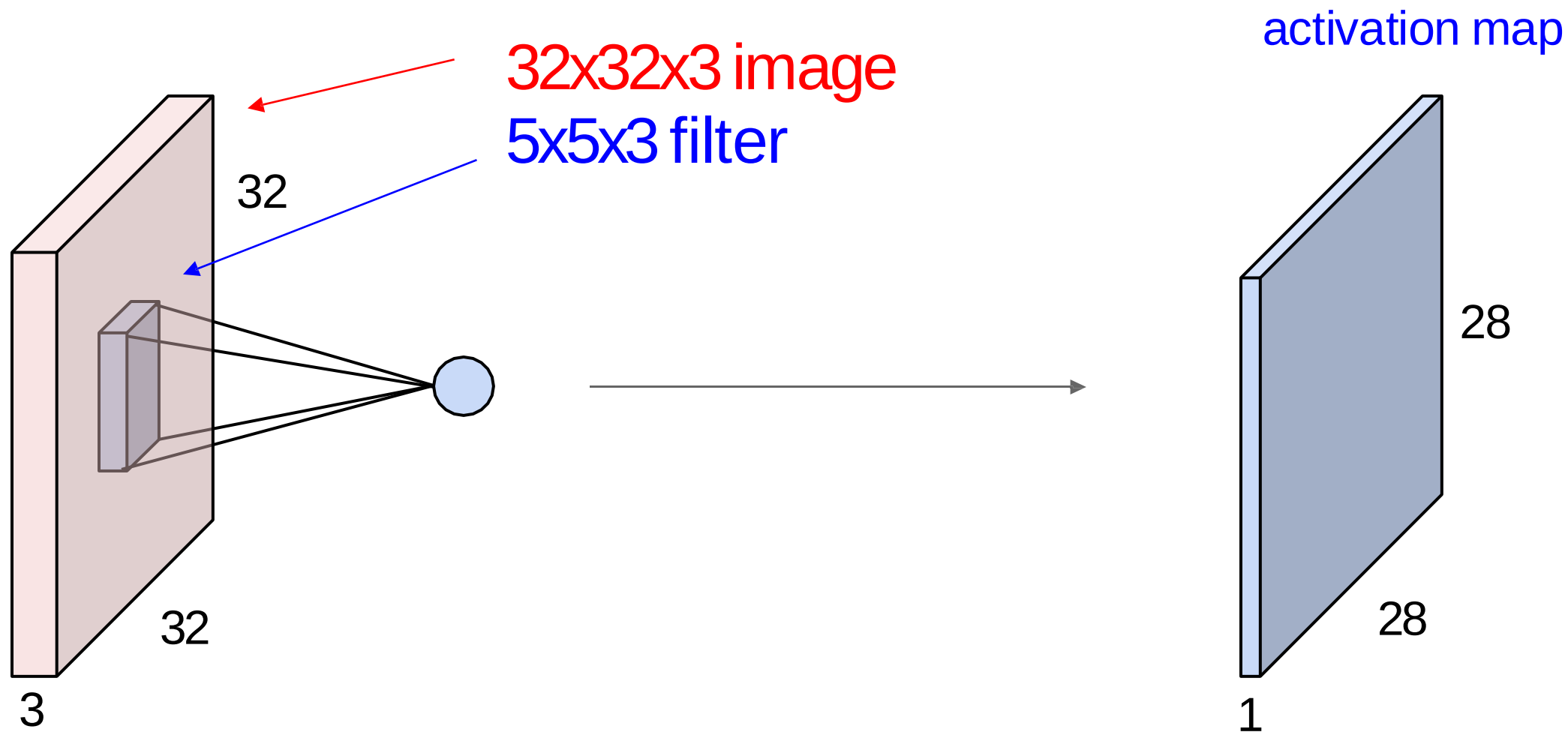
■ 问题：一层卷积处理输入视频可能不够



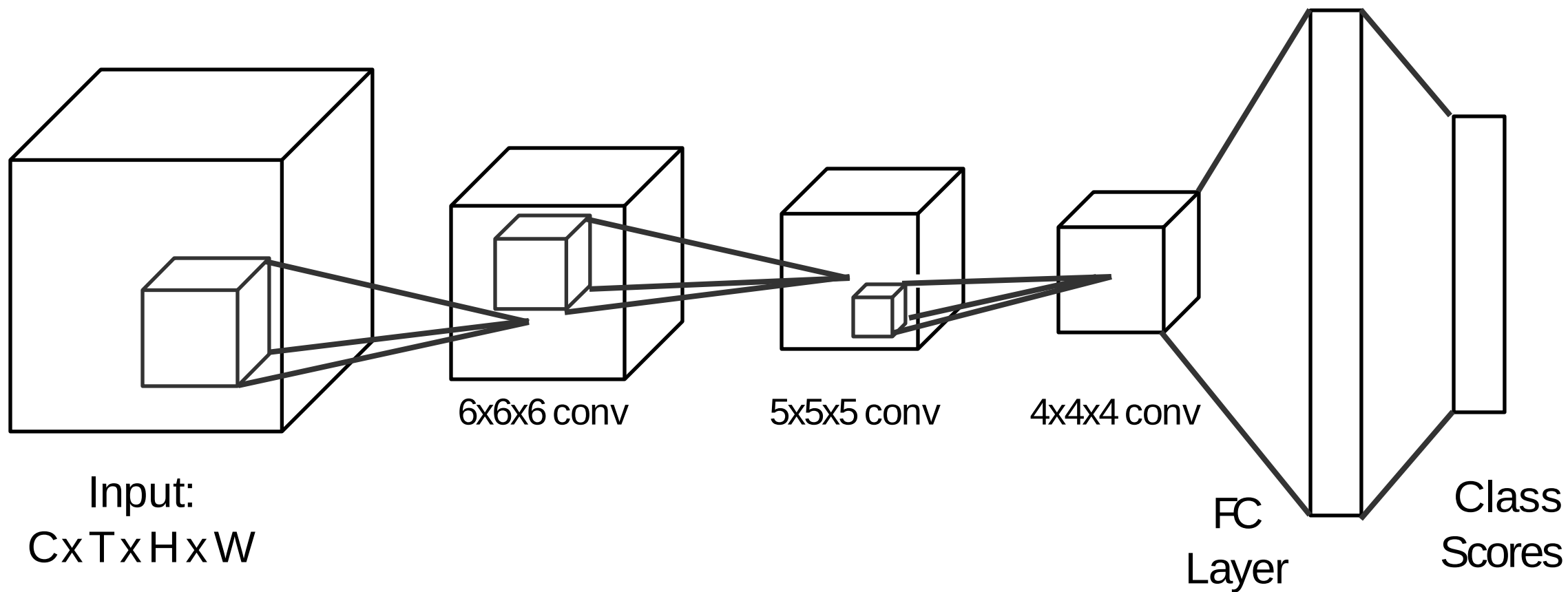
■ 3D 卷积



■ 2D 卷积



■ 3D 卷积



■ Early Fusion v.s. Late Fusion v.s. 3D CNN

Late
Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3

■ Early Fusion v.s. Late Fusion v.s. 3D CNN

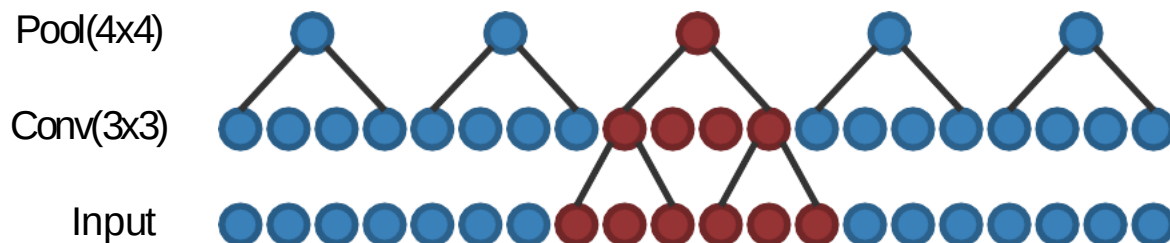
Late
Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3



■ Early Fusion v.s. Late Fusion v.s. 3D CNN

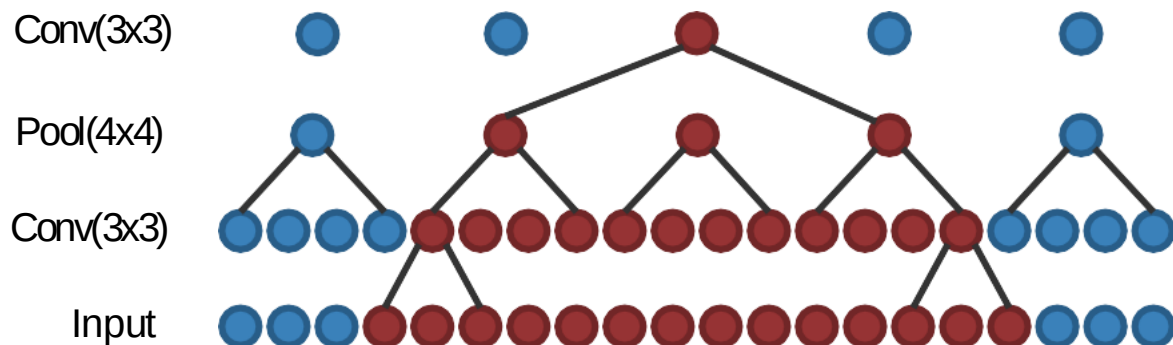
	Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Late Fusion	Input	3 x 20 x 64 x 64	
	Conv2D(3x3, 3→12)	12 x 20 x 64 x 64	1 x 3 x 3
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6



■ Early Fusion v.s. Late Fusion v.s. 3D CNN

Late
Fusion

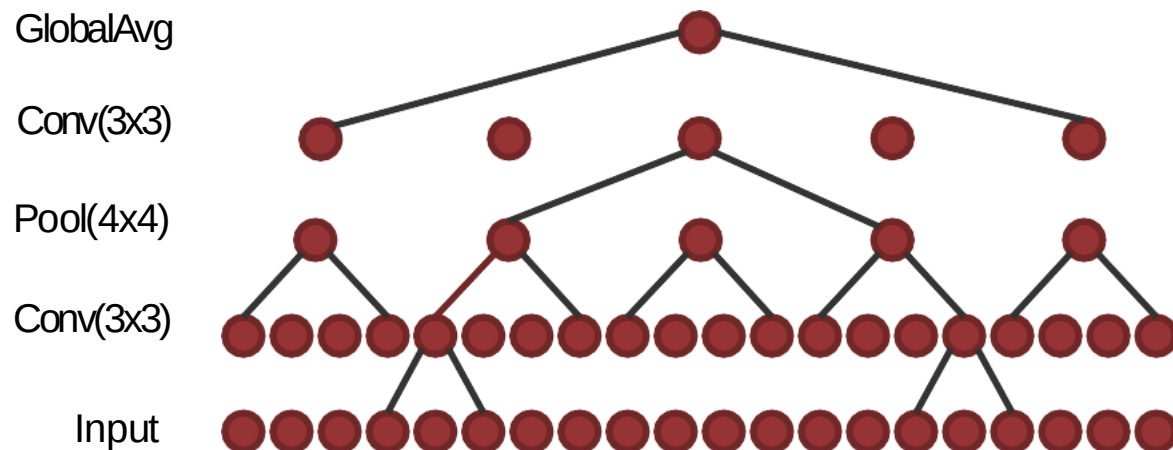
Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14



■ Early Fusion v.s. Late Fusion v.s. 3D CNN

Late
Fusion

Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64



	Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Late Fusion	Input	3 x 20 x 64 x 64	
	Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
	Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
	GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64
Early Fusion	Input	3 x 20 x 64 x 64	
	Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3
	Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6
	Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64
3D CNN	Input	3 x 20 x 64 x 64	
	Conv3D(3x3x3, 3->12)	12 x 20 x 64 x 64	3 x 3 x 3
	Pool3D(4x4x4)	12 x 5 x 16 x 16	6 x 6 x 6
	Conv3D(3x3x3, 12->24)	24 x 5 x 16 x 16	14 x 14 x 14
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64

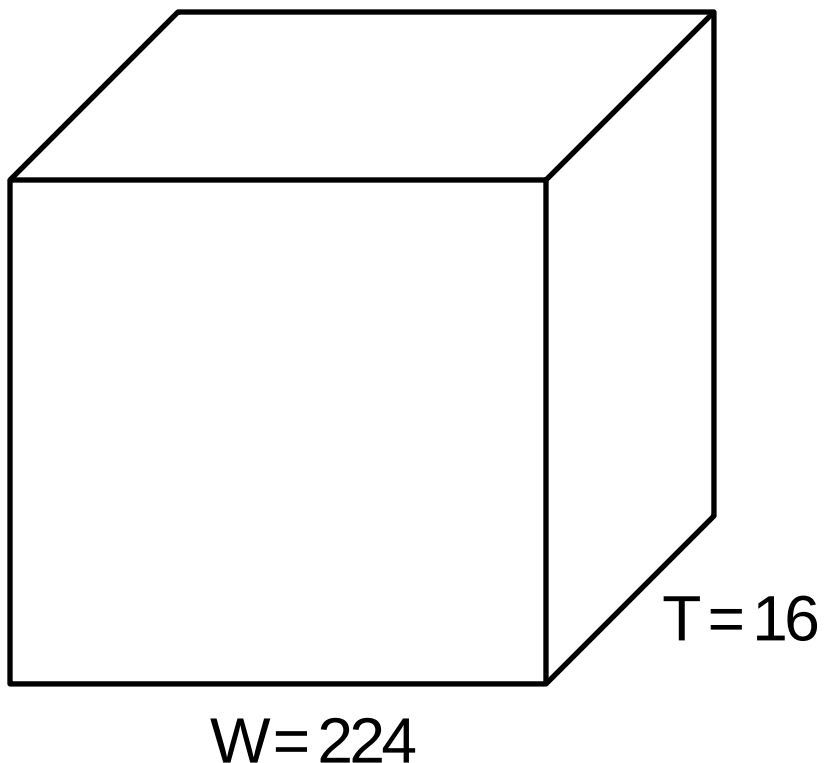
	Layer	Size (C x T x H x W)	Receptive Field (T x H x W)
Late Fusion	Input	3 x 20 x 64 x 64	
	Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
	Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
	GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64
Early Fusion	Input	3 x 20 x 64 x 64	
	Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3
	Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6
	Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64
3D CNN	Input	3 x 20 x 64 x 64	
	Conv3D(3x3x3, 3->12)	12 x 20 x 64 x 64	3 x 3 x 3
	Pool3D(4x4x4)	12 x 5 x 16 x 16	6 x 6 x 6
	Conv3D(3x3x3, 12->24)	24 x 5 x 16 x 16	14 x 14 x 14
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64

它们的区别是什么？

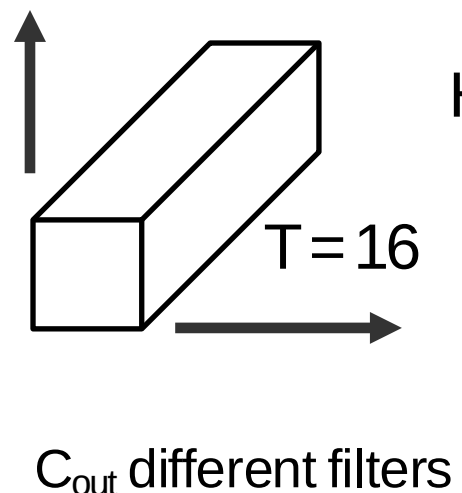
→ 有参考帧

■ 2D Conv (Early Fusion)

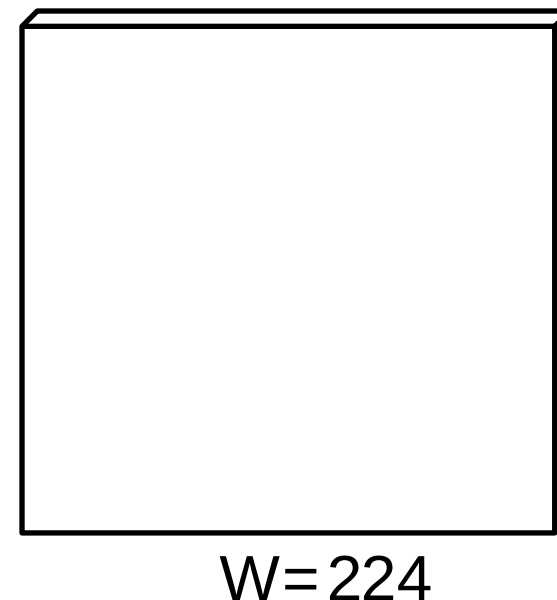
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



Weight:
 $C_{out} \times C_{in} \times (T \times 3 \times 3)$
Slide over x and y

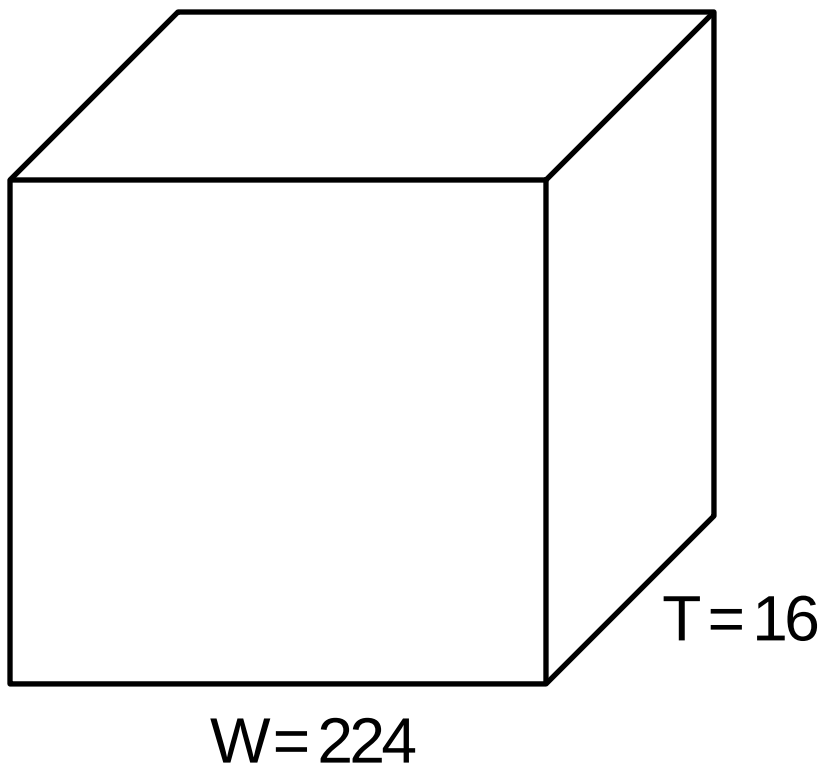


Output:
 $C_{out} \times H \times W$
2D grid with C_{out} -dim
feat at each point

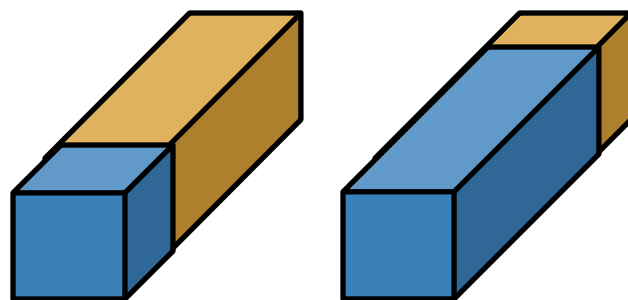


■ 2D Conv (Early Fusion)

Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



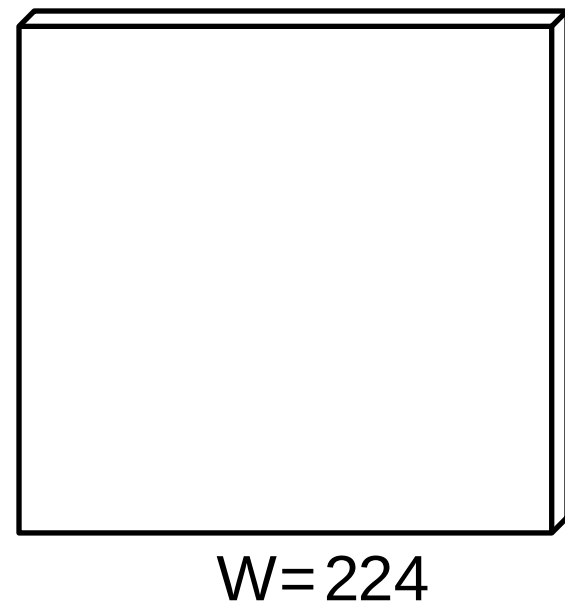
Weight:
 $C_{out} \times C_{in} \times (T \times 3 \times 3)$
Slide over x and y



C_{out} different filters

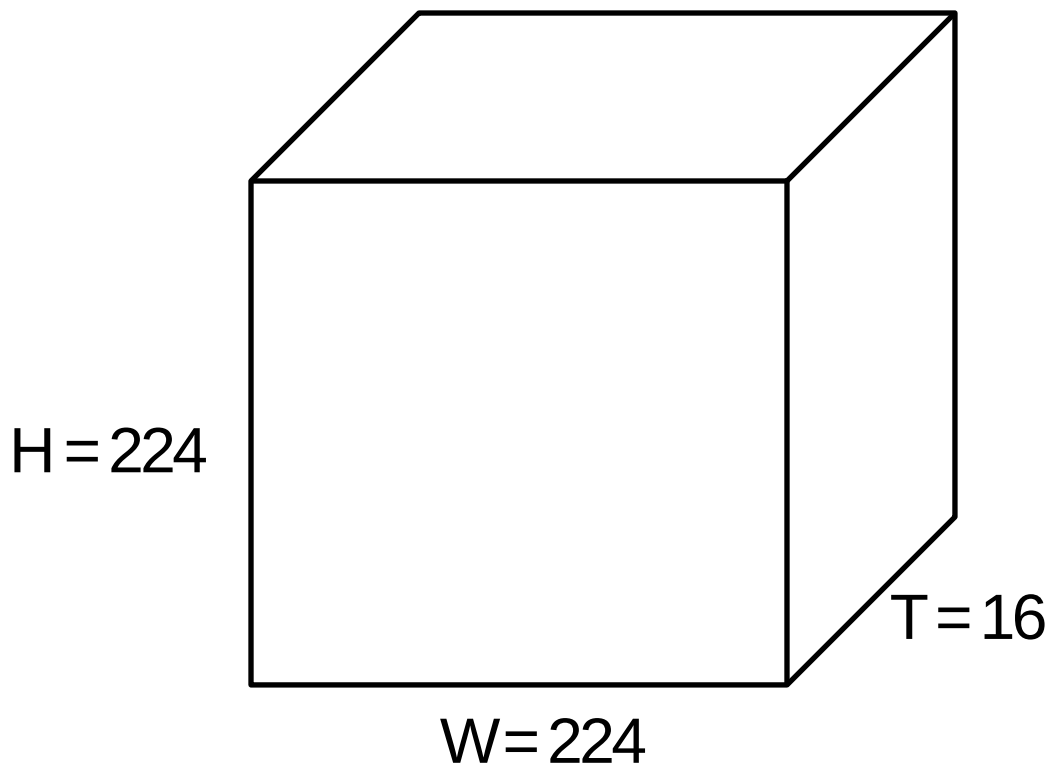
没有时间平移不变性!
不同时间都有单独滤波器

Output:
 $C_{out} \times H \times W$
2D grid with C_{out} -dim
feat at each point

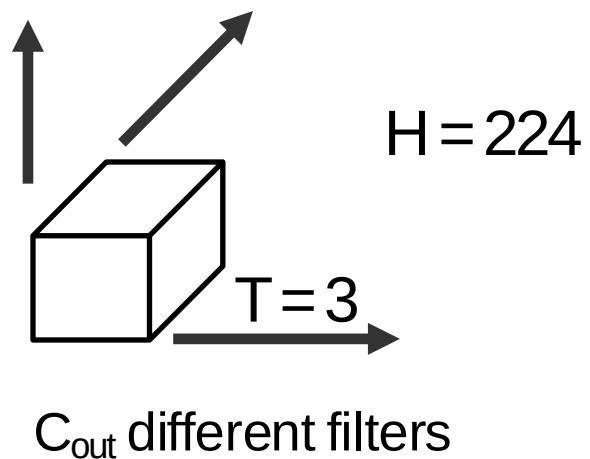


■ 3D Conv

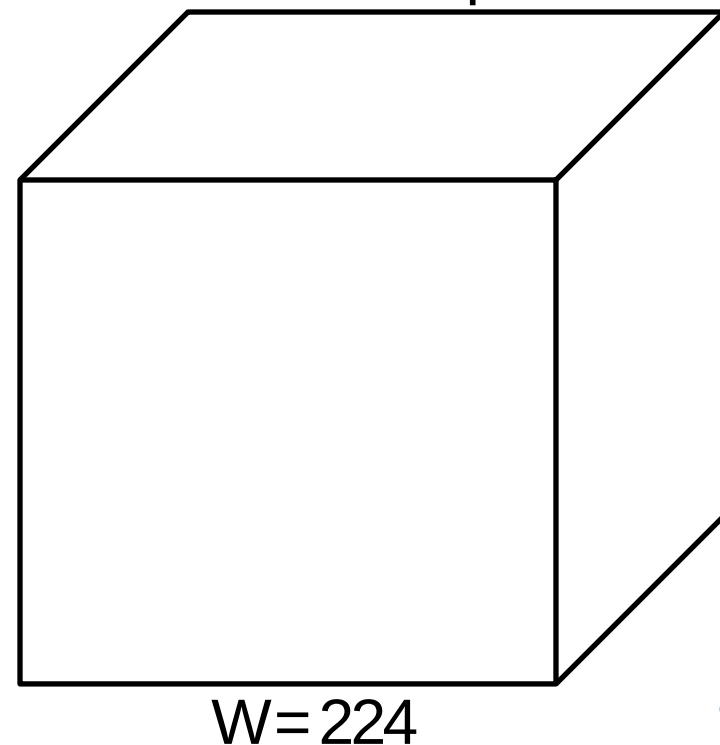
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



Weight:
 $C_{out} \times C_{in} \times (3 \times 3 \times 3)$
Slide over x and y

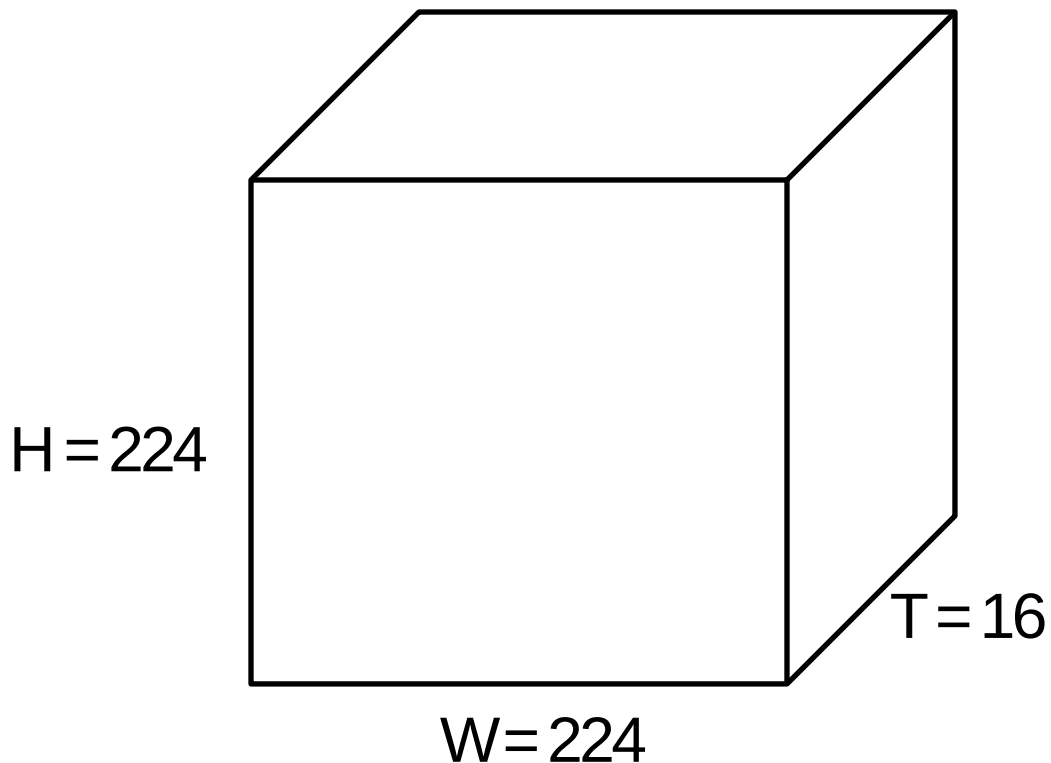


Output:
 $C_{out} \times T \times H \times W$
3D grid with C_{out} -dim
feat at each point

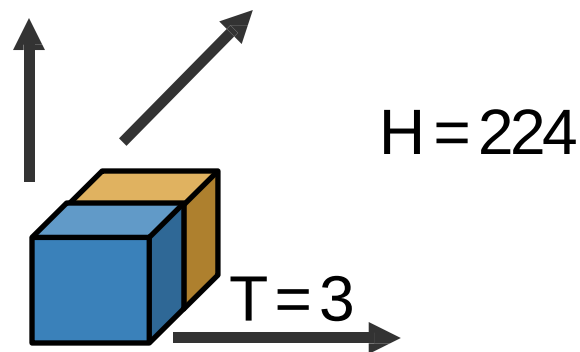


3D Conv

Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)

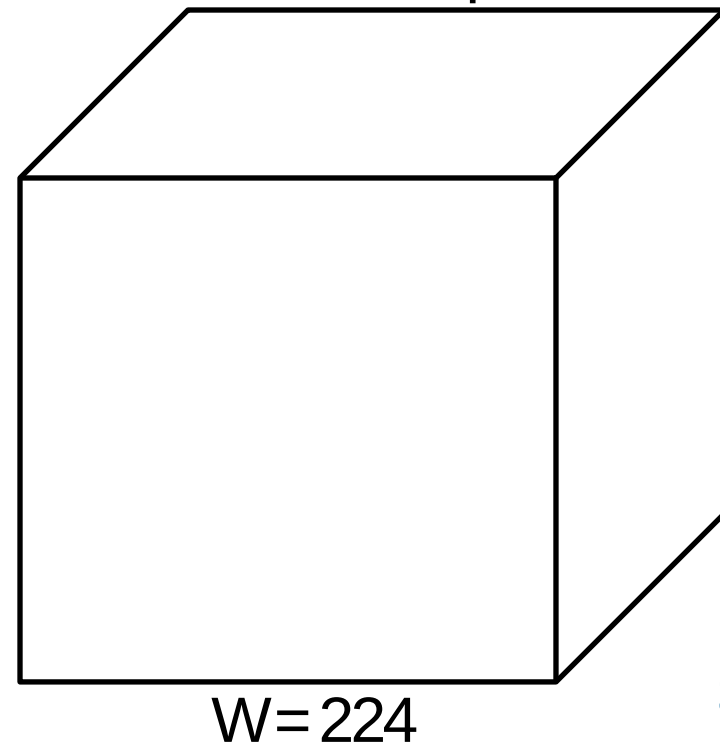


Weight:
 $C_{out} \times C_{in} \times (3 \times 3 \times 3)$
Slide over x and y



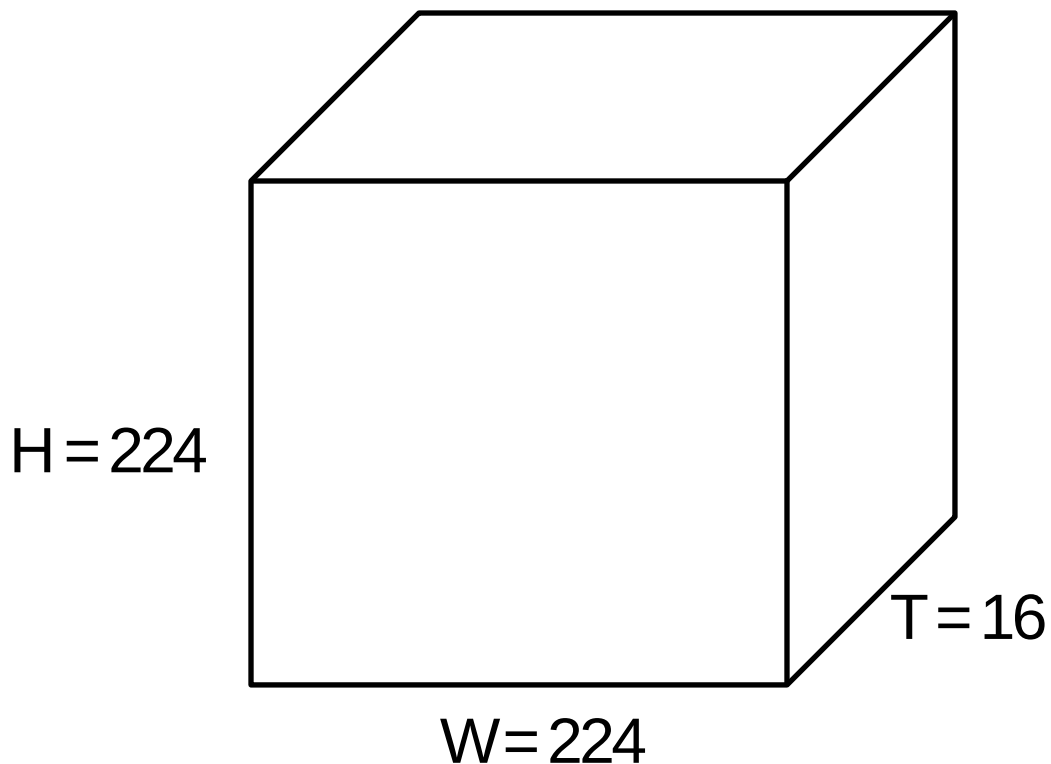
C_{out} different filters
具有时间平移不变性!

Output:
 $C_{out} \times T \times H \times W$
3D grid with C_{out} -dim
feat at each point



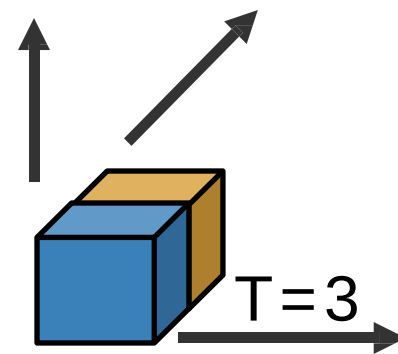
3D Conv

Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



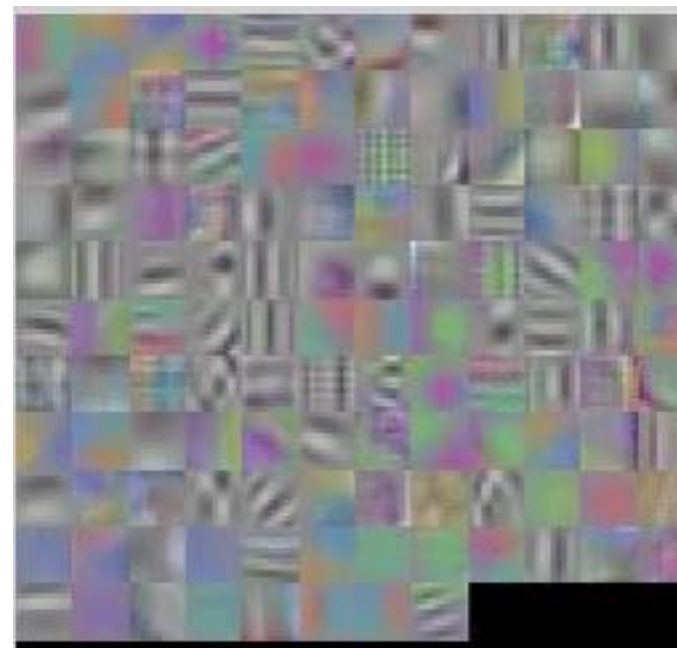
Weight:

$C_{out} \times C_{in} \times (3 \times 3 \times 3)$
Slide over x and y



C_{out} different filters
具有时间平移不变性!

第一层滤波器的形状为
3 (RGB) $\times$ 4 (帧) $\times$ 5 $\times$ 5 (空间)



■ 视频数据集：Sports-1M



track cycling
cycling
track cycling
road bicycle racing
marathon
ultramarathon



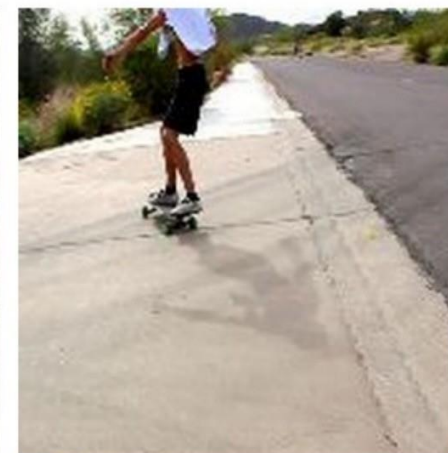
ultramarathon
ultramarathon
half marathon
running
marathon
inline speed skating



heptathlon
heptathlon
decathlon
hurdles
pentathlon
sprint (running)



bikejoring
mushing
bikejoring
harness racing
skijoring
carting



longboarding
longboarding
aggressive inline skating
freestyle scootering
freeboard (skateboard)
sandboarding

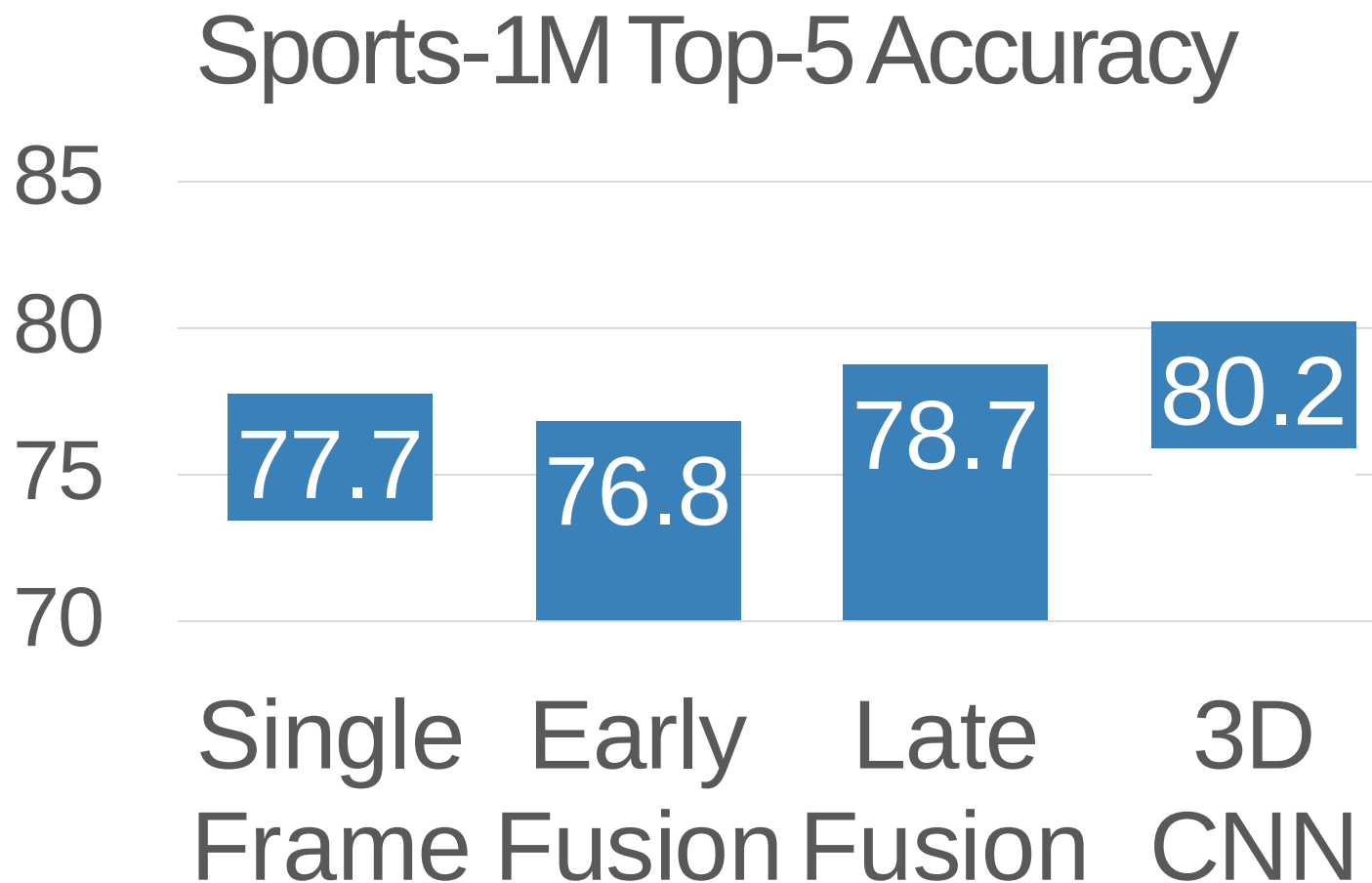
100万个带有487种不同运动
标签的YouTube视频

Ground Truth

Correct prediction

Incorrect prediction

■ Early Fusion v.s. Late Fusion v.s. 3D CNN



单帧模型效果很好

3D模型提升很多

- C3D: 3D CNN 中的 VGG
 - 只使用 3x3x3 conv 和池化
 - 在Sports-1M上预训练

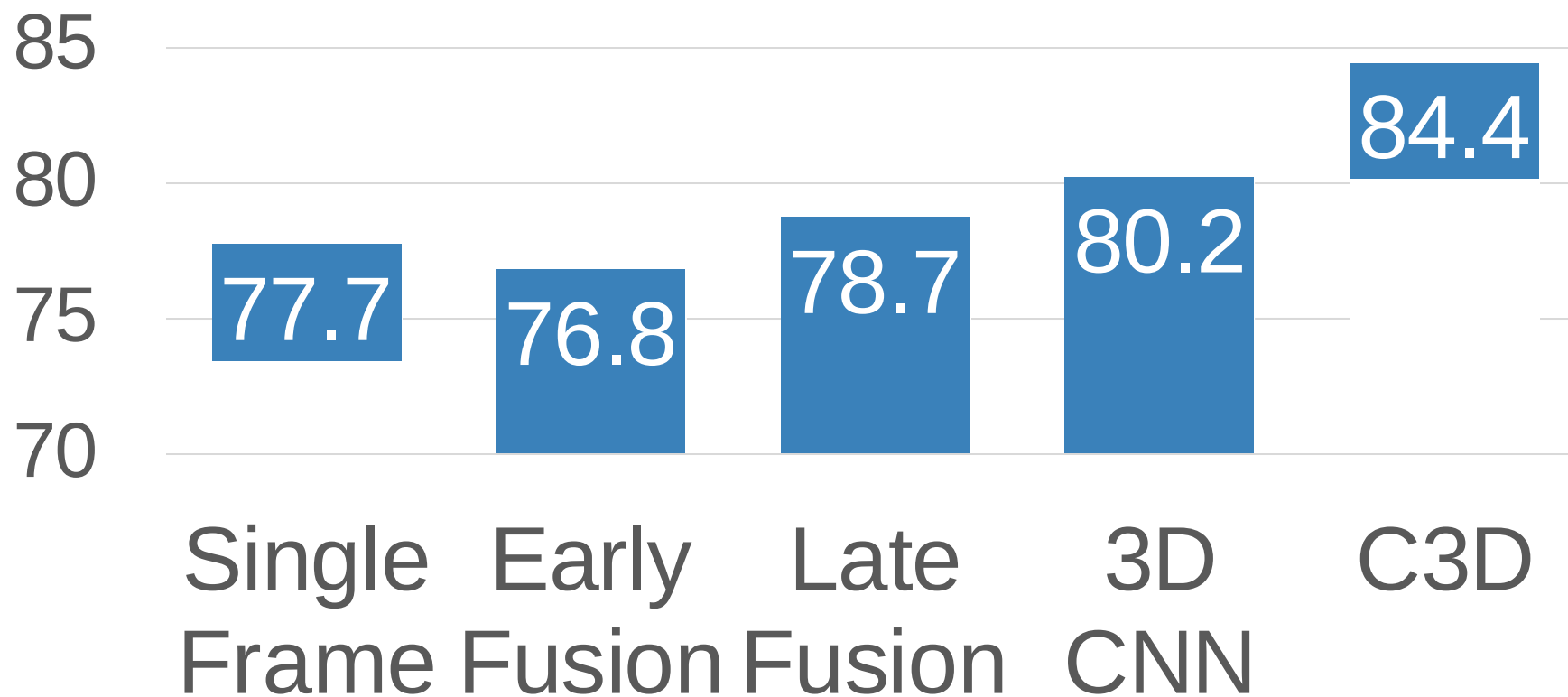
Layer	Size
Input	3 x 16 x 112 x 112
Conv1 (3x3x3)	64 x 16 x 112 x 112
Pool1 (1x2x2)	64 x 16 x 56 x 56
Conv2 (3x3x3)	128 x 16 x 56 x 56
Pool2 (2x2x2)	128 x 8 x 28 x 28
Conv3a (3x3x3)	256 x 8 x 28 x 28
Conv3b (3x3x3)	256 x 8 x 28 x 28
Pool3 (2x2x2)	256 x 4 x 14 x 14
Conv4a (3x3x3)	512 x 4 x 14 x 14
Conv4b (3x3x3)	512 x 4 x 14 x 14
Pool4 (2x2x2)	512 x 2 x 7 x 7
Conv5a (3x3x3)	512 x 2 x 7 x 7
Conv5b (3x3x3)	512 x 2 x 7 x 7
Pool5	512 x 1 x 3 x 3
FC6	4096
FC7	4096
FC8	C

- **C3D: 3D CNN 中的 VGG**
 - 只使用 3x3x3 conv 和池化
 - 在Sports-1M上预训练
- **问题: 3x3x3 conv 计算量很大**
 - AlexNet: 0.7 GFLOP
 - VGG-16: 13.6 GFLOP
 - C3D: 39.5 GFLOP (2.9x VGG!)

Layer	Size	MFLOPs
Input	3 x 16 x 112 x 112	
Conv1 (3x3x3)	64 x 16 x 112 x 112	1.04
Pool1 (1x2x2)	64 x 16 x 56 x 56	
Conv2 (3x3x3)	128 x 16 x 56 x 56	11.10
Pool2 (2x2x2)	128 x 8 x 28 x 28	
Conv3a (3x3x3)	256 x 8 x 28 x 28	5.55
Conv3b (3x3x3)	256 x 8 x 28 x 28	11.10
Pool3 (2x2x2)	256 x 4 x 14 x 14	
Conv4a (3x3x3)	512 x 4 x 14 x 14	2.77
Conv4b (3x3x3)	512 x 4 x 14 x 14	5.55
Pool4 (2x2x2)	512 x 2 x 7 x 7	
Conv5a (3x3x3)	512 x 2 x 7 x 7	0.69
Conv5b (3x3x3)	512 x 2 x 7 x 7	0.69
Pool5	512 x 1 x 3 x 3	
FC6	4096	0.51
FC7	4096	0.45
FC8	C	0.05

■ Early Fusion v.s. Late Fusion v.s. 3D CNN

Sports-1M Top-5 Accuracy



■ 衡量运动 (Motion) : 光流 (Optical Flow)

Image at frame t

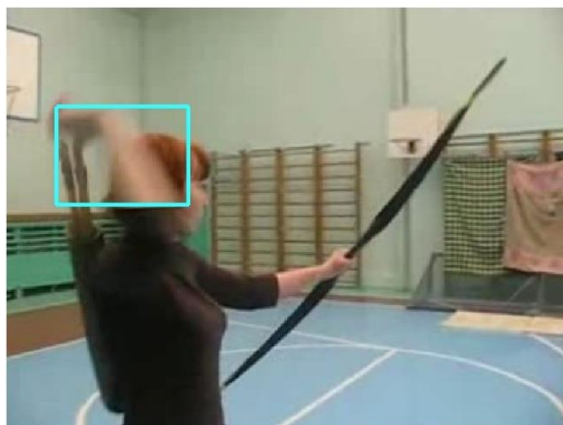
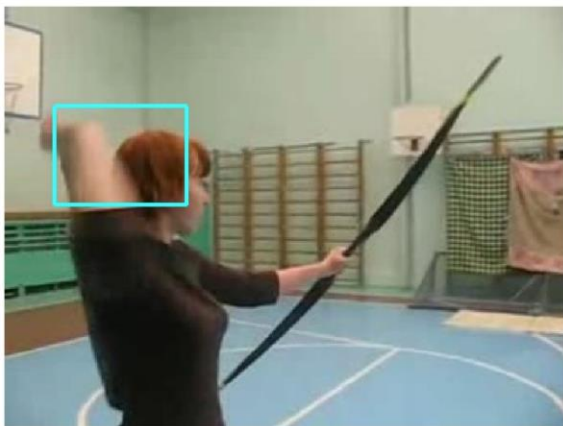
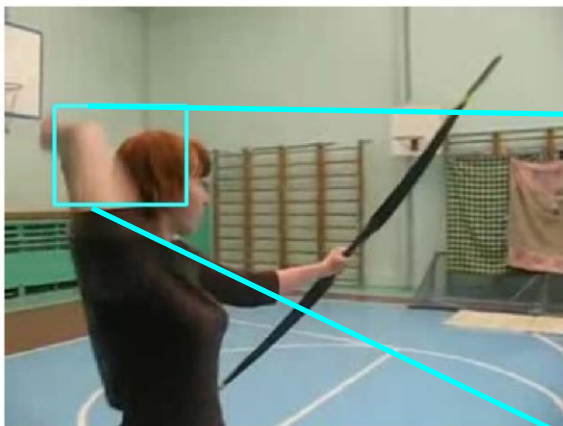


Image at frame $t+1$

■ 衡量运动 (Motion) : 光流 (Optical Flow)

Image at frame t



光流在图像 I_t 和 I_{t+1} 之间产生位移场 F

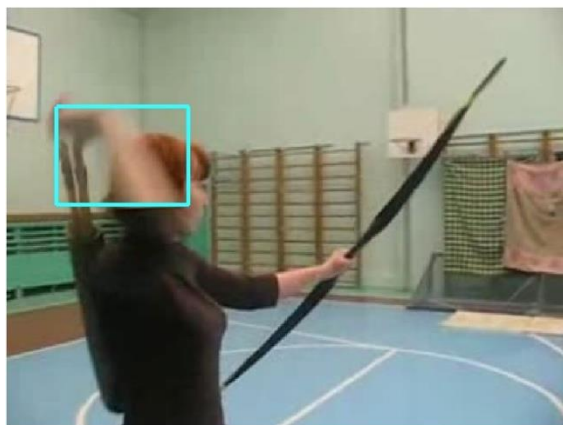
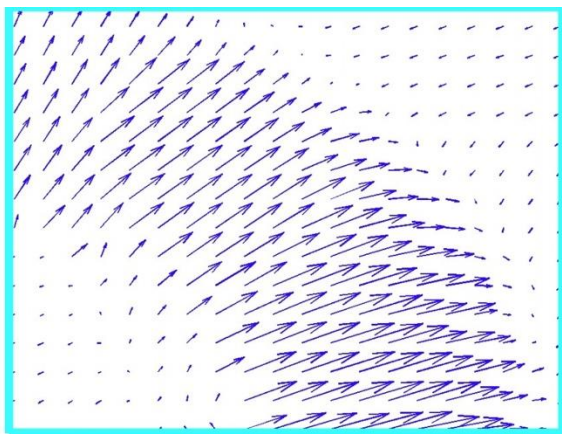


Image at frame $t+1$

每个像素会在下一帧移动到哪个位置:

$$F(x, y) = (dx, dy)$$

$$I_{t+1}(x+dx, y+dy) = I_t(x, y)$$

■ 衡量运动 (Motion) : 光流 (Optical Flow)

Image at frame t

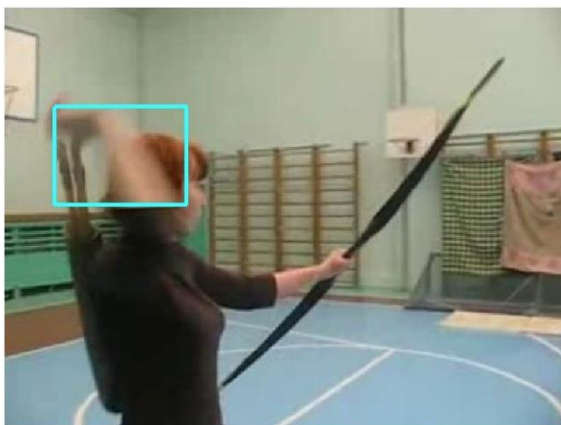
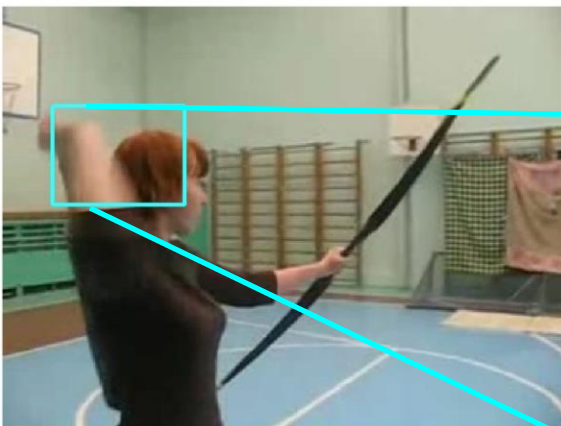
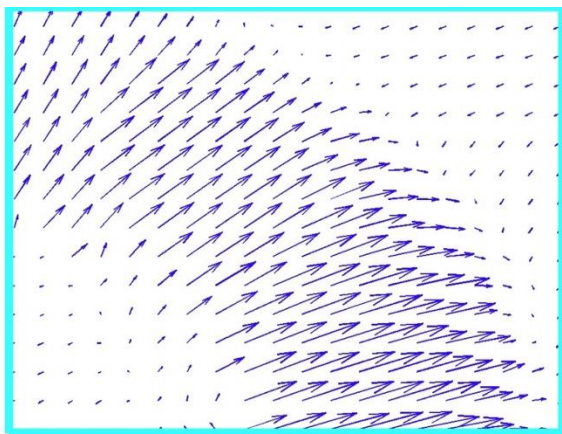


Image at frame $t+1$

光流在图像 I_t 和 I_{t+1} 之间产生位移场 F



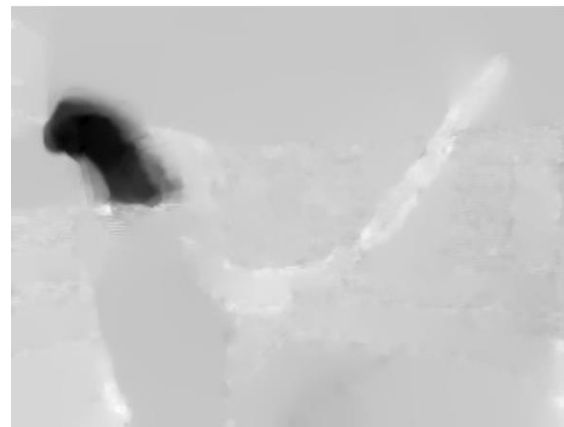
每个像素会在下一帧移动到哪个位置:

$$F(x, y) = (dx, dy)$$

$$I_{t+1}(x+dx, y+dy) = I_t(x, y)$$

光流可以突出局部运动

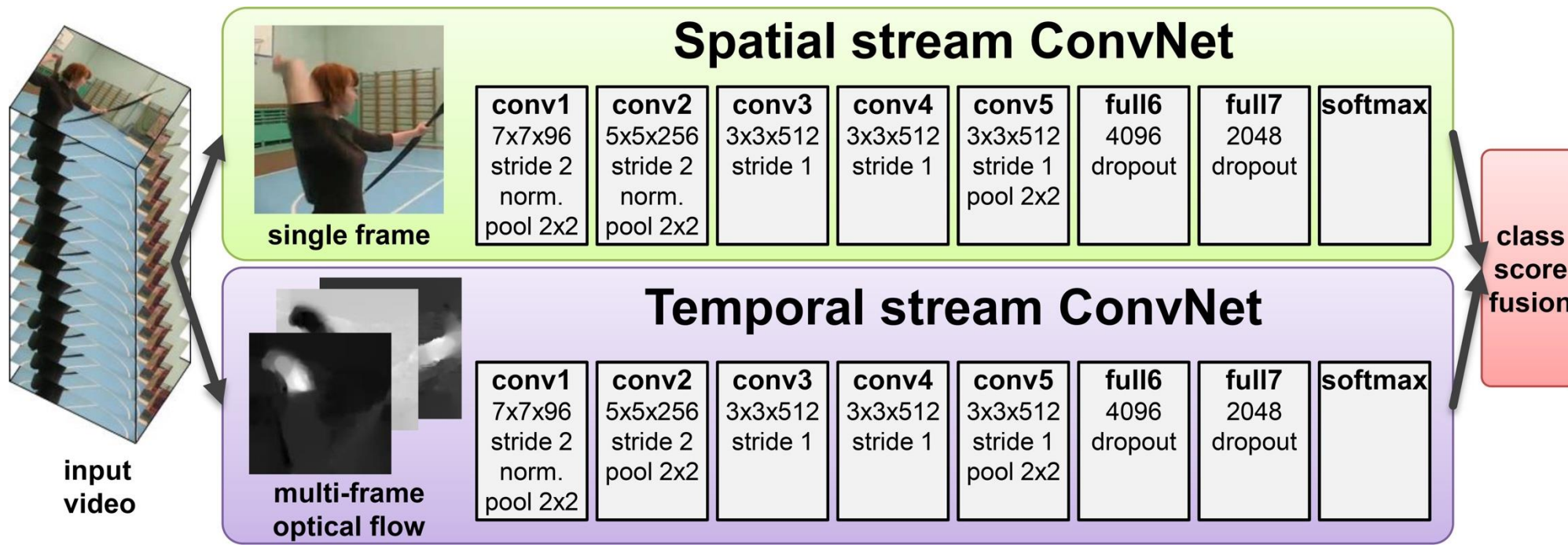
Horizontal flow dx



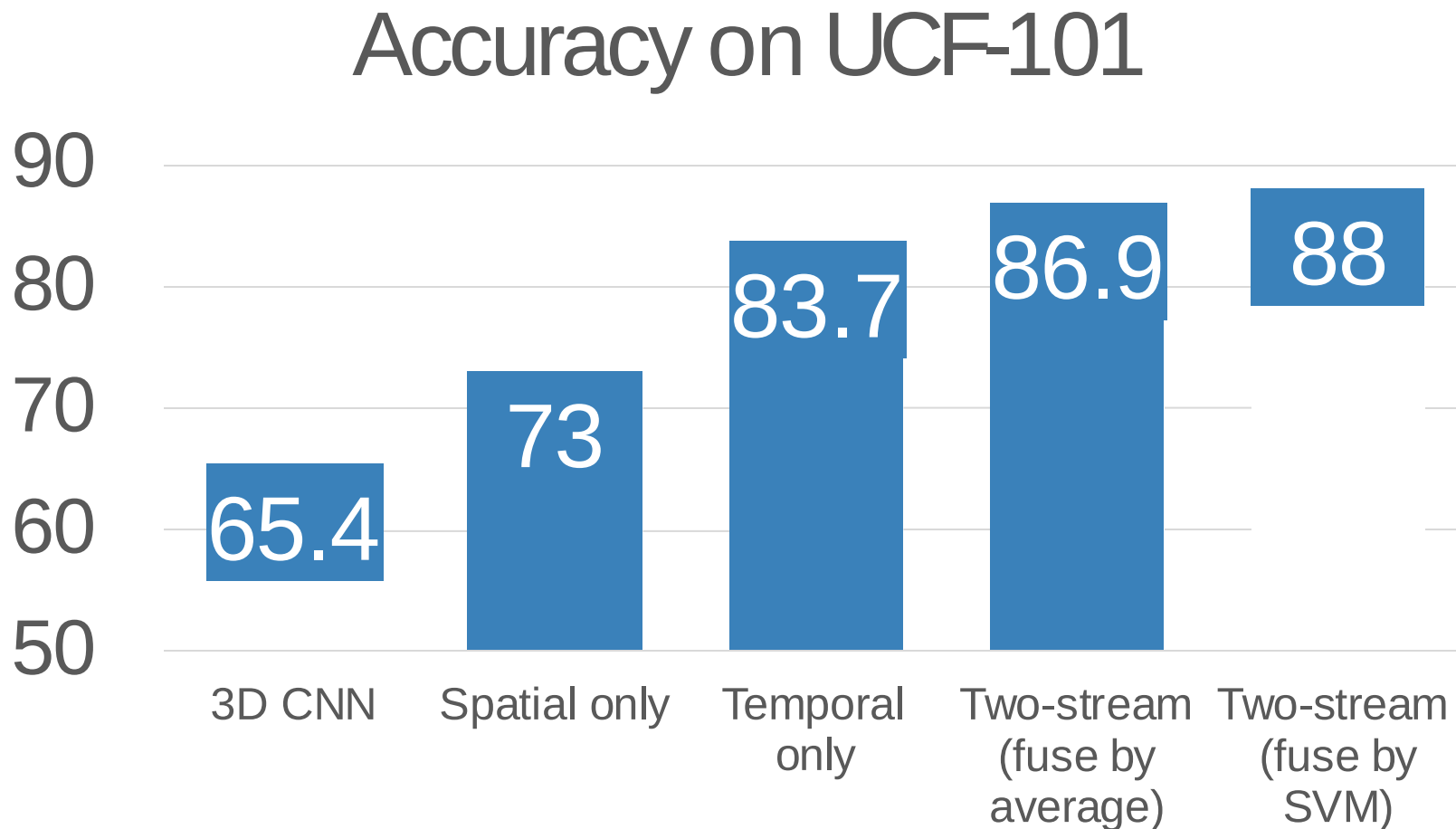
Vertical Flow dy

■ 分别处理 motion 和 appearance: Two-Stream Networks

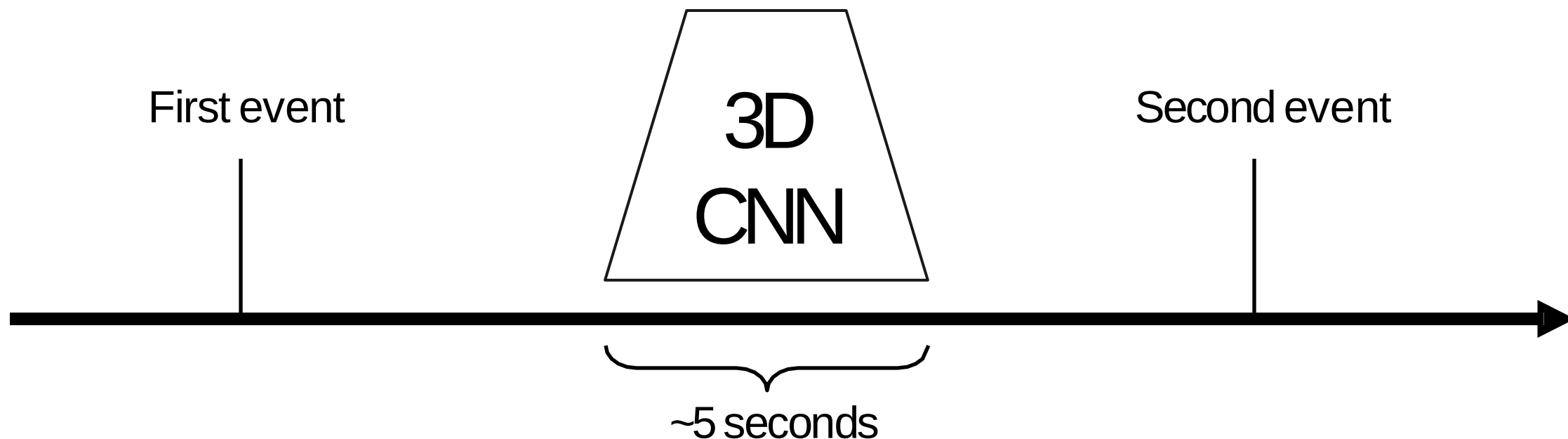
Input: Single Image
 $3 \times H \times W$



■ 分别处理 motion 和 appearance: Two-Stream Networks

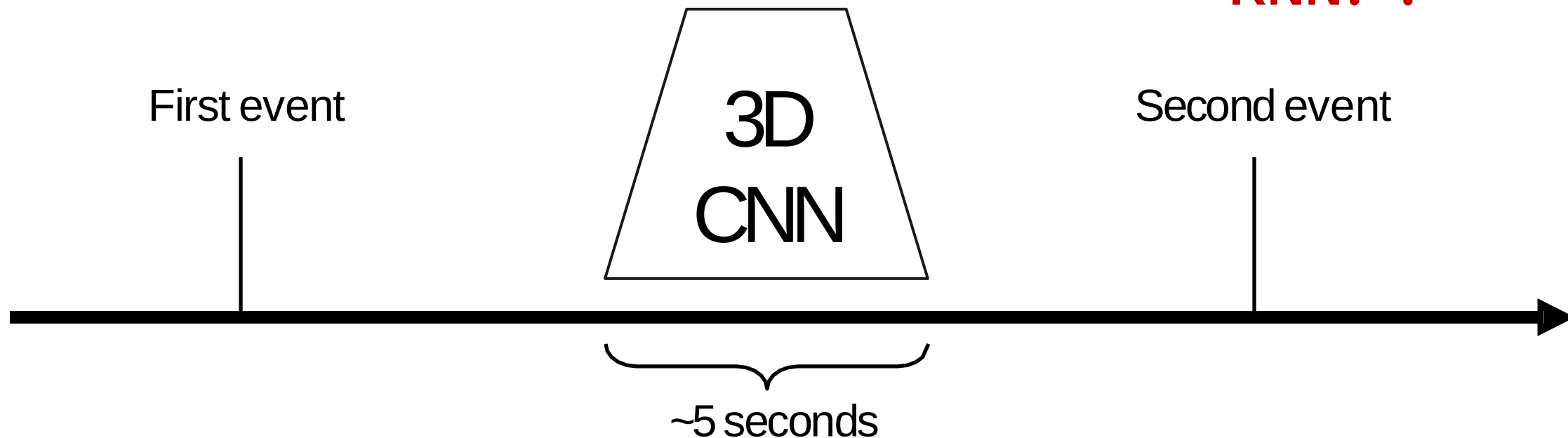


- 建模长时间时序结构
- 目前为止，我们主要关注约2-5秒的非常短的片段中模拟帧之间的局部运动。如何处理长时间时序结构呢？

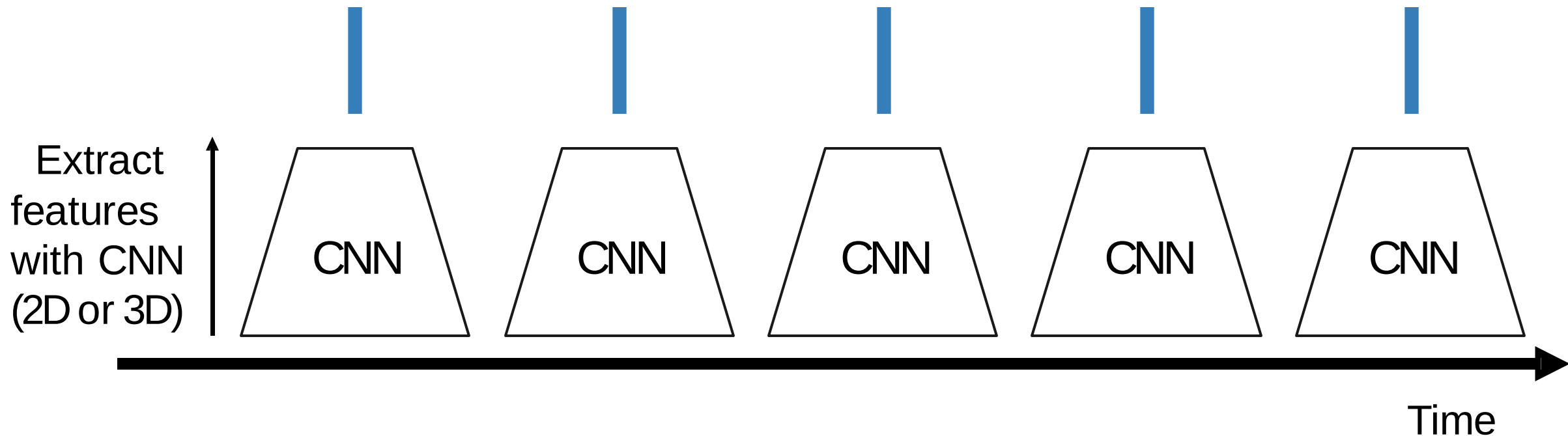


- 建模长时间时序结构
- 目前为止，我们主要关注约2-5秒的非常短的片段中模拟帧之间的局部运动。如何处理长时间时序结构呢？

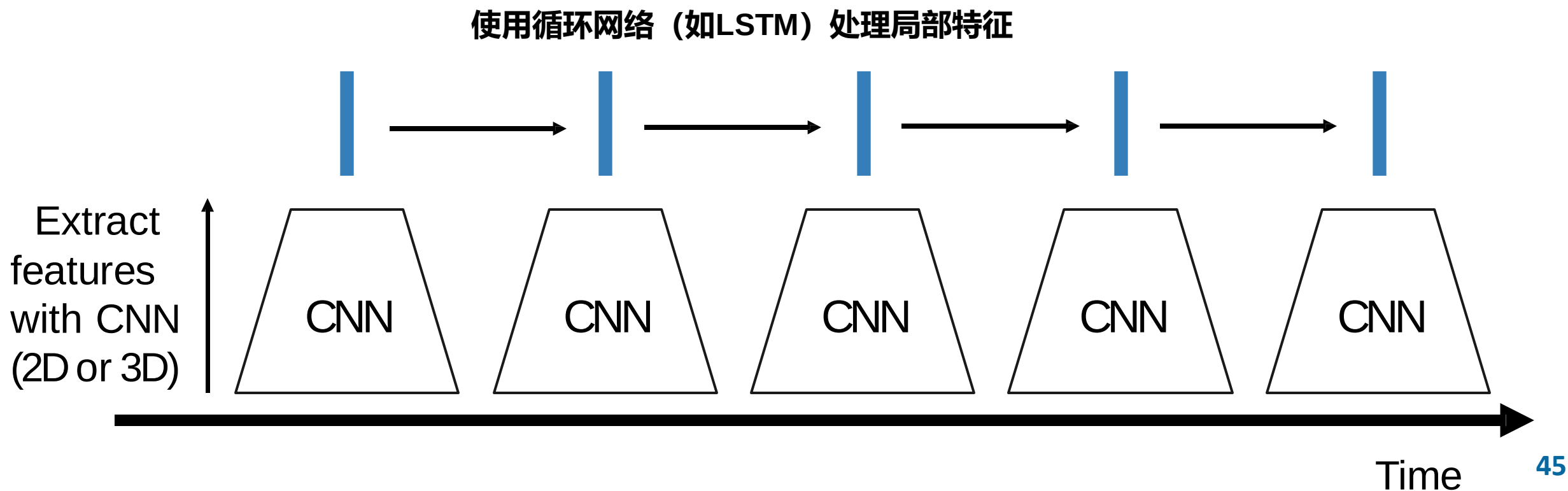
RNN? !



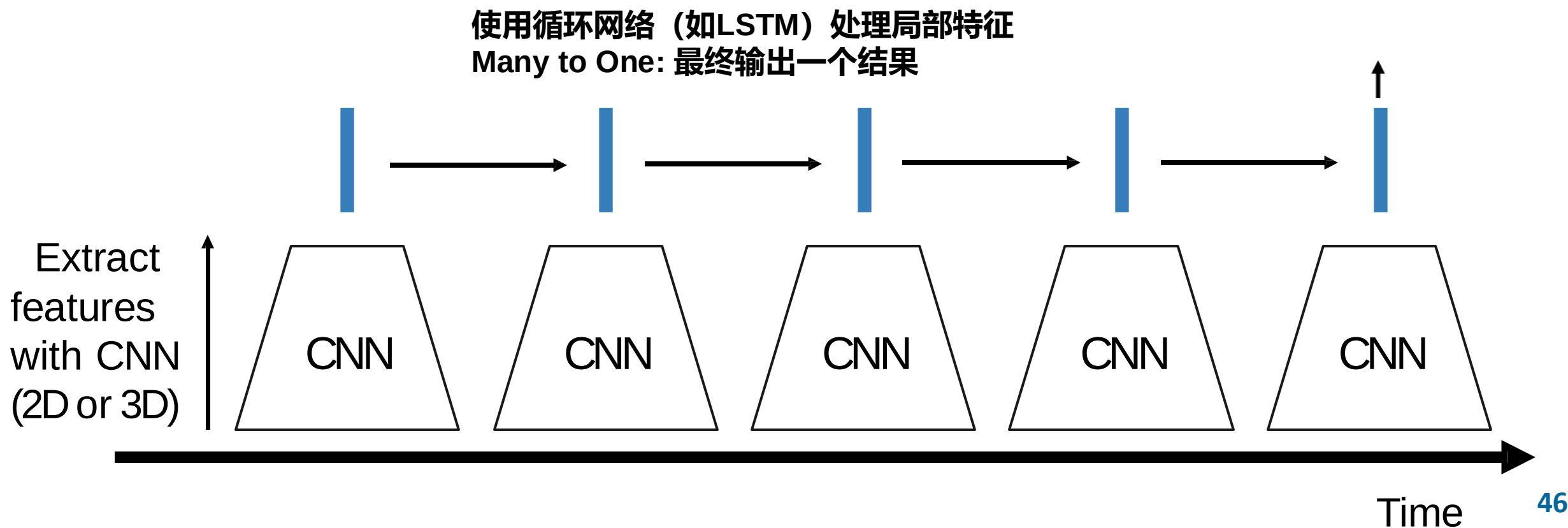
■ 建模长时间时序结构



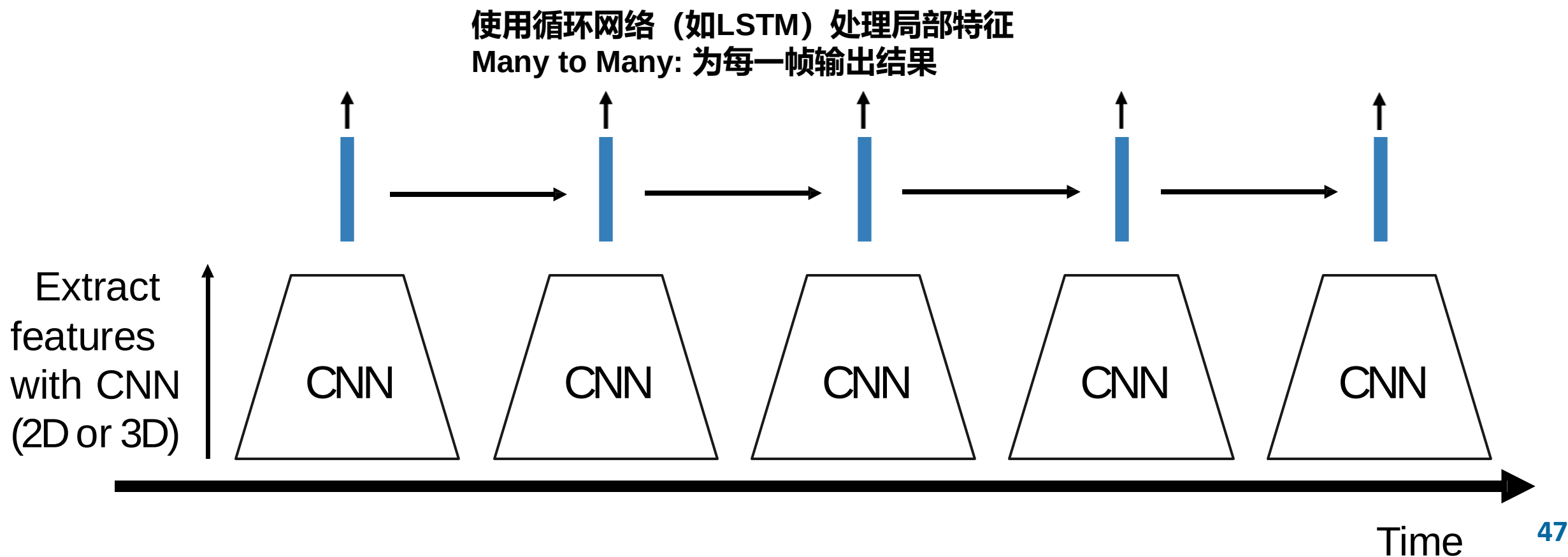
■ 建模长时间时序结构



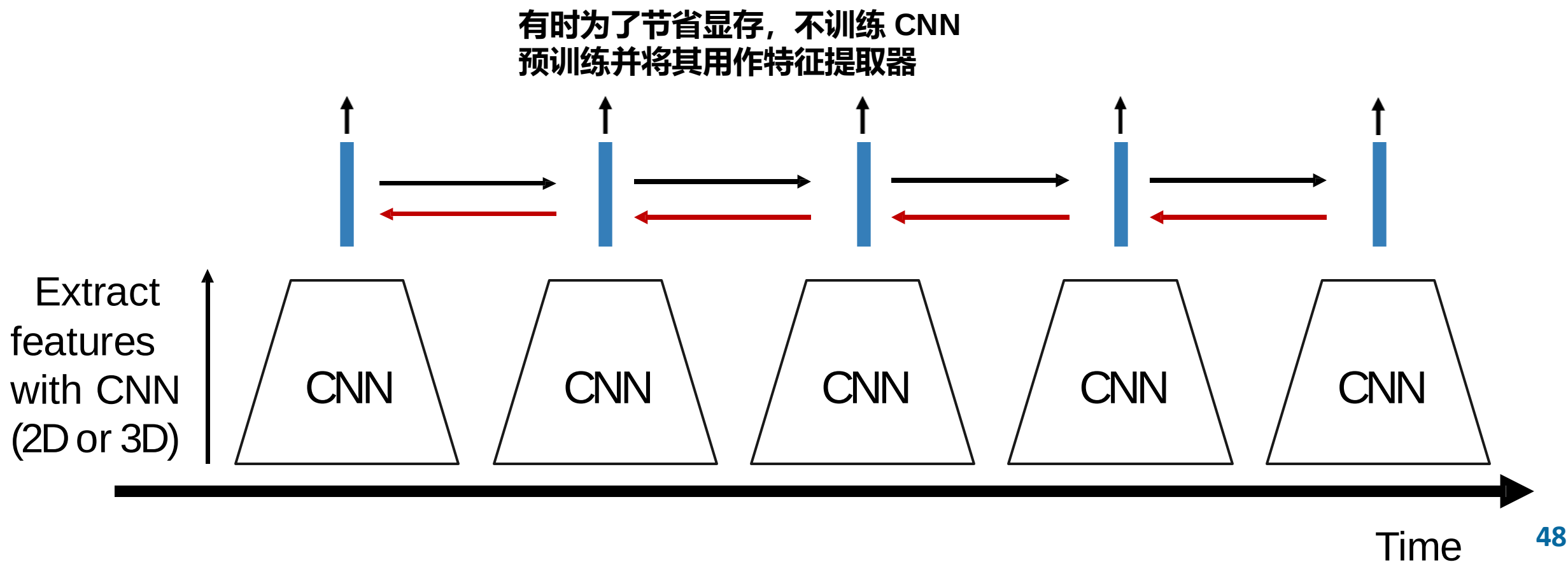
■ 建模长时间时序结构



■ 建模长时间时序结构



■ 建模长时间时序结构

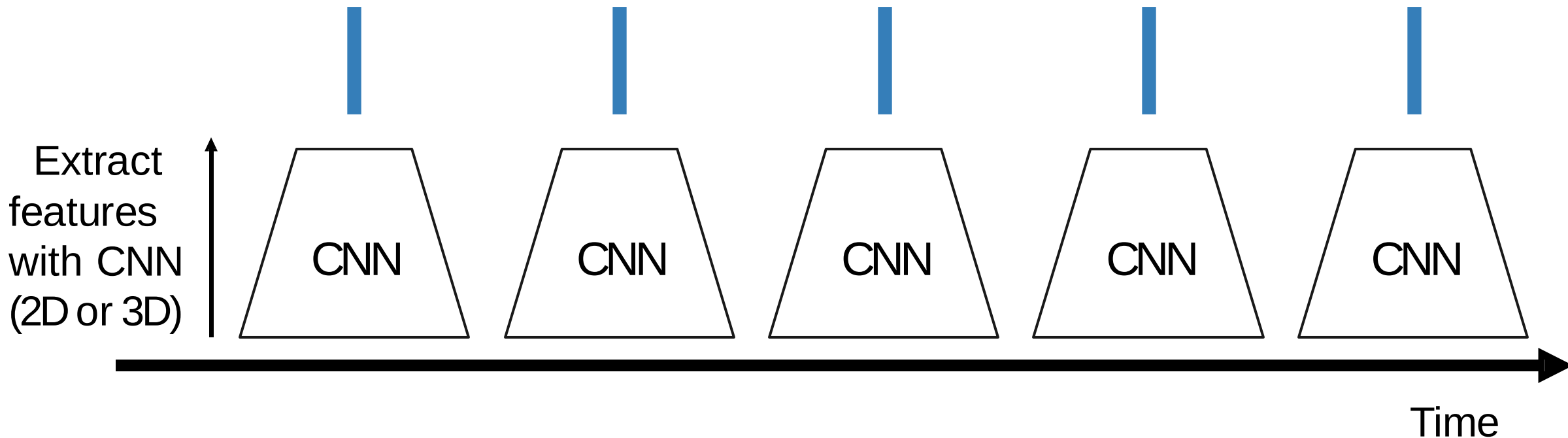


■ 建模长时间时序结构

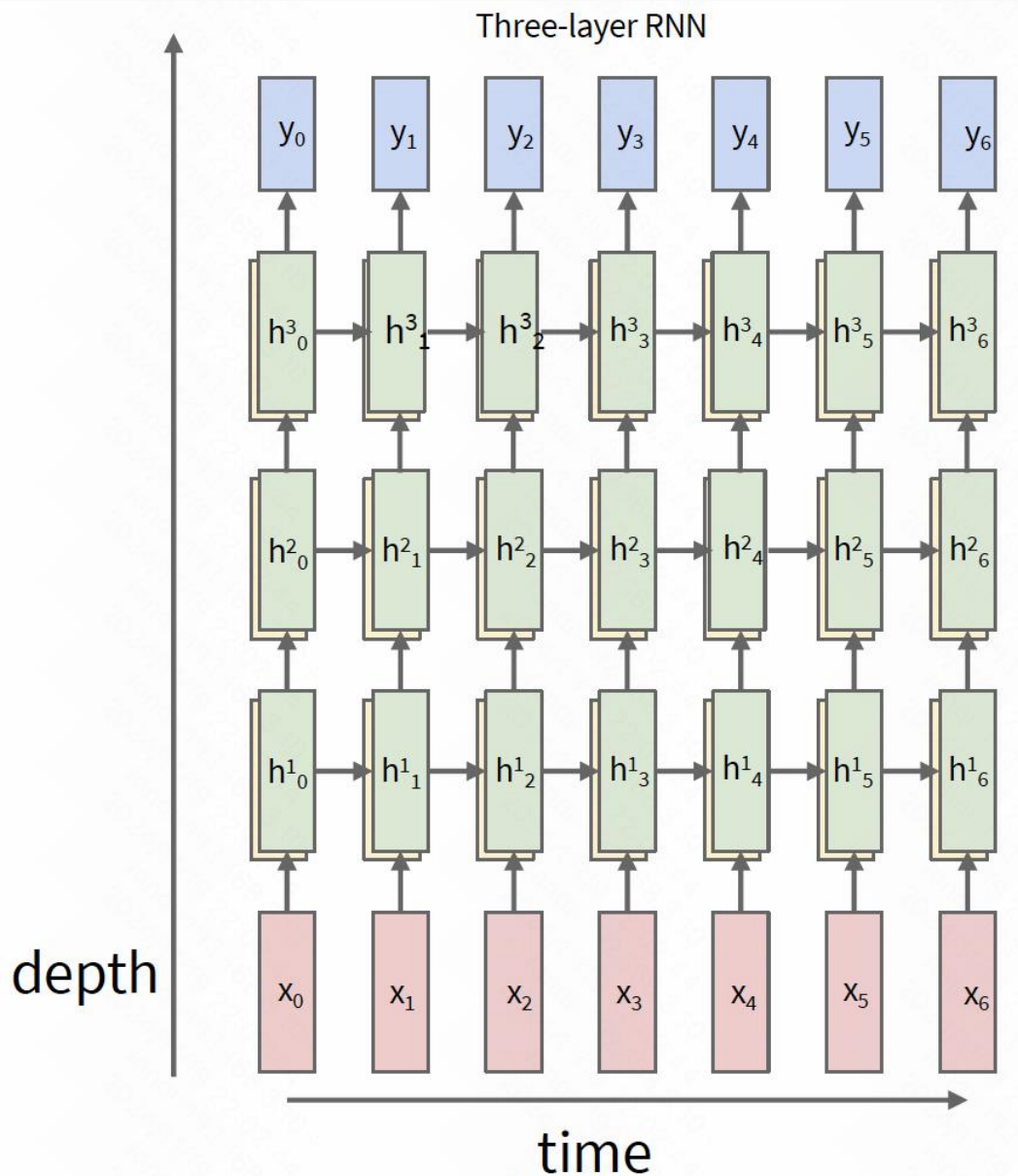
CNN内部：每个特征值都是固定时间窗口的函数（局部时间结构）

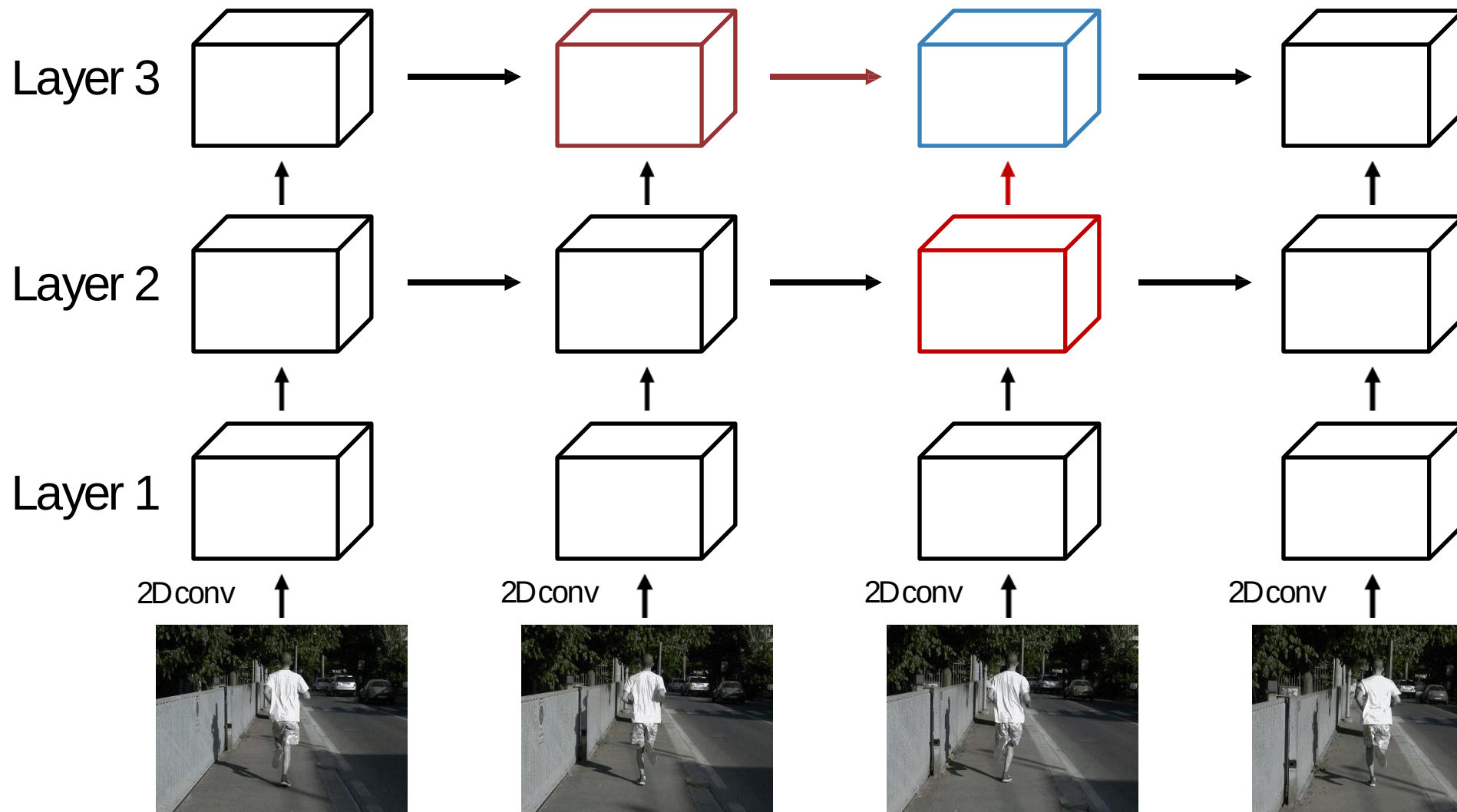
RNN内部：每个向量都是所有先前向量的函数（全局时间结构）

我们可以结合这两种方法吗？



- 回顾：多层 RNN
- 我们可以使用类似的结构来处理视频





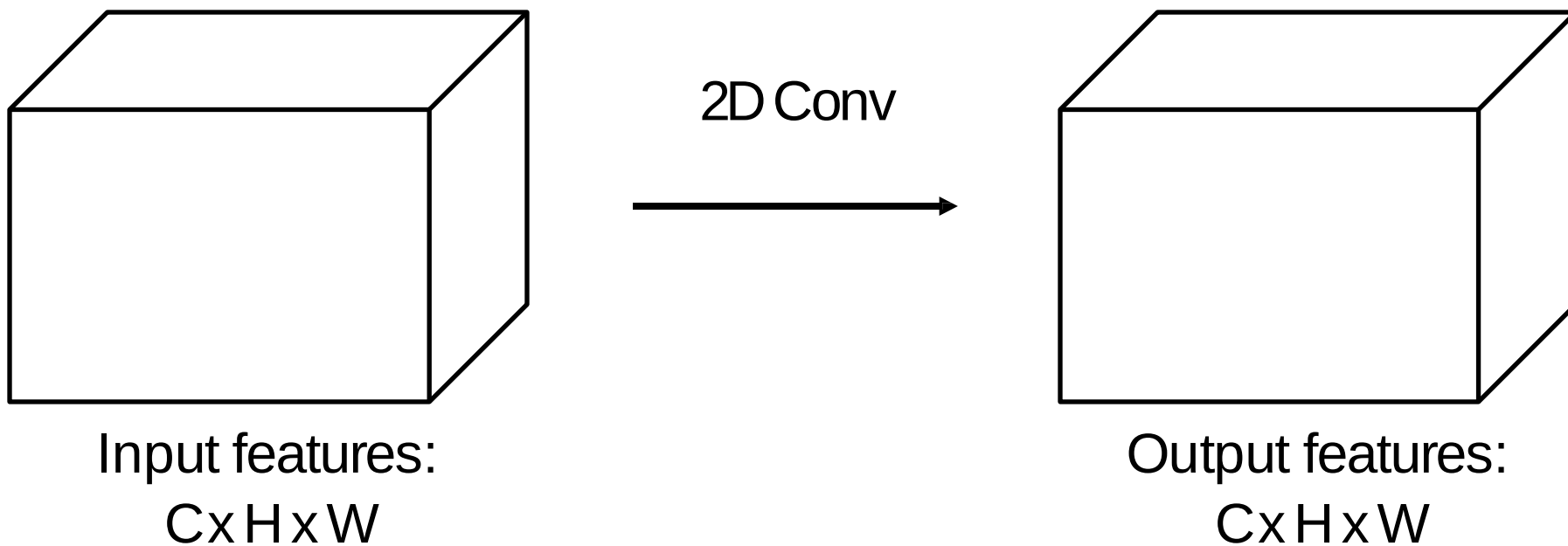
整个网络处理
二维特征图：
 $C \times H \times W$

每个输出都取决于两个输入：
同一层，前一时间步
上一层，相同时间步

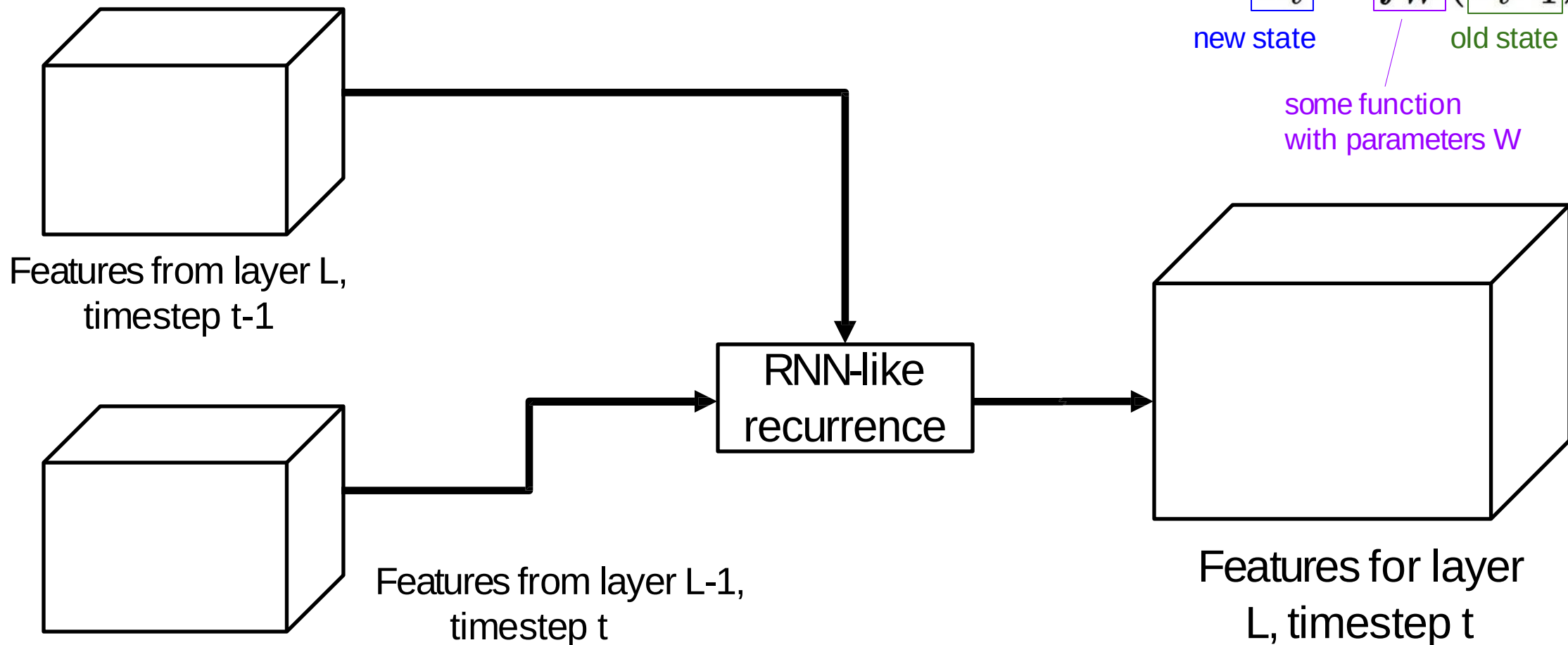
在每一层使用不同的权重，跨时间共享权重

■ Recurrent Convolutional Network

Normal 2D CNN:



■ Recurrent Convolutional Network



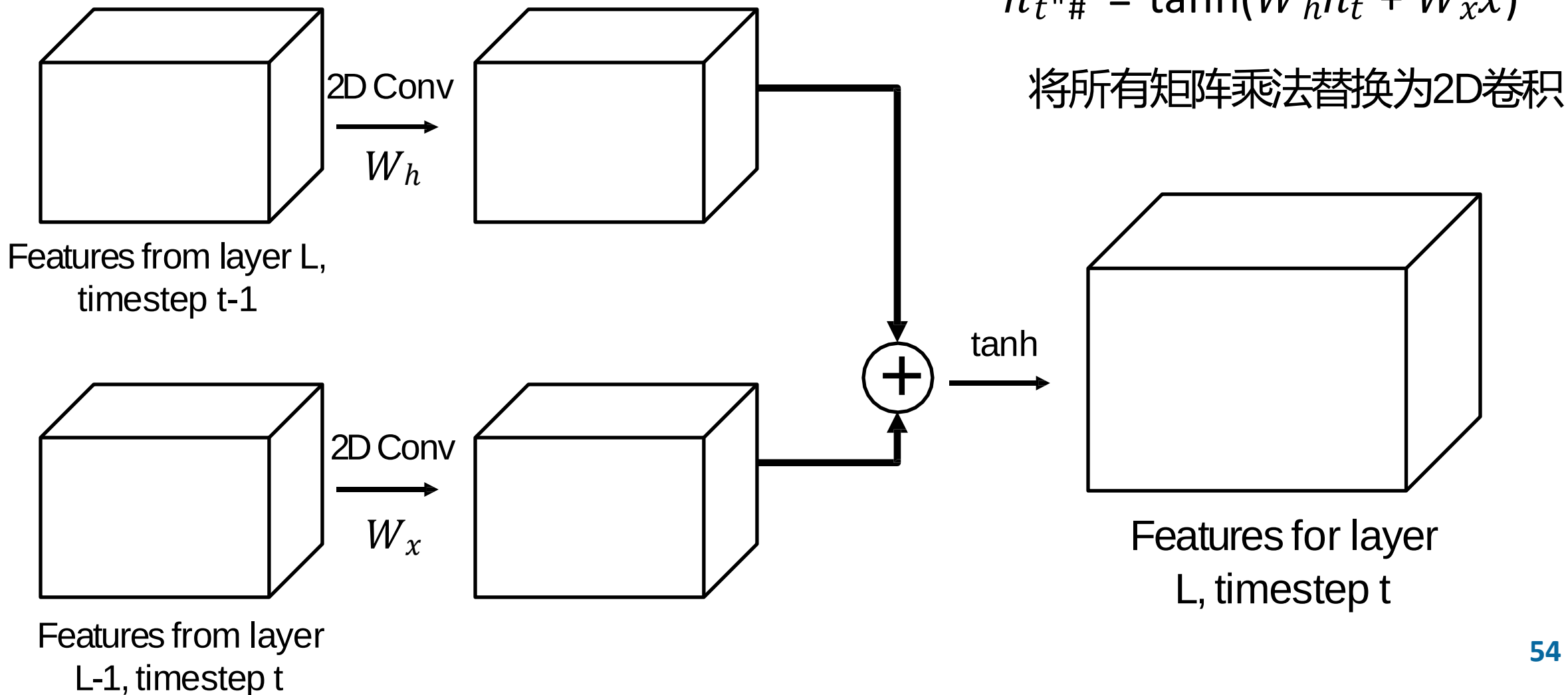
回顾: RNN

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state / old state

some function
with parameters W

■ Recurrent Convolutional Network



回顾: Vanilla RNN

$$h_{t+1} = \tanh(W_h h_t + W_x x_t)$$

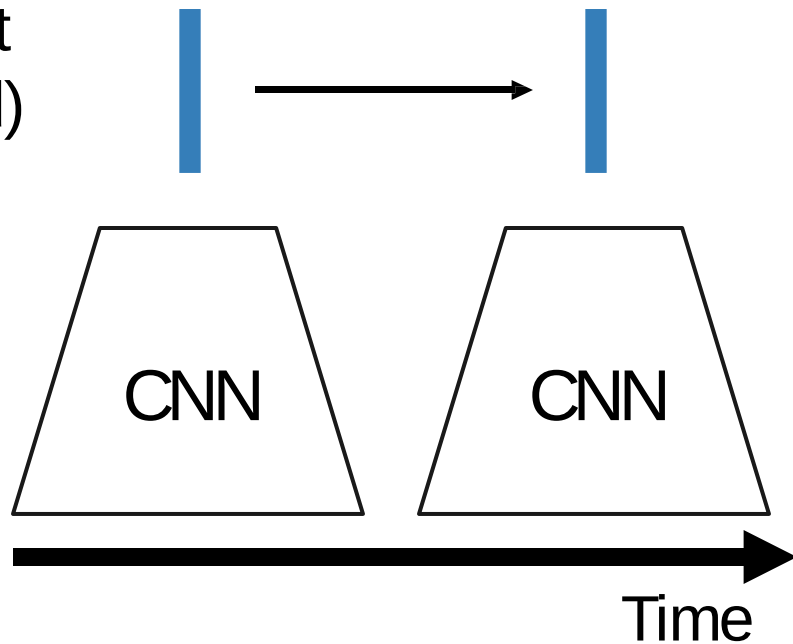
将所有矩阵乘法替换为2D卷积

■ 建模长时间时序结构

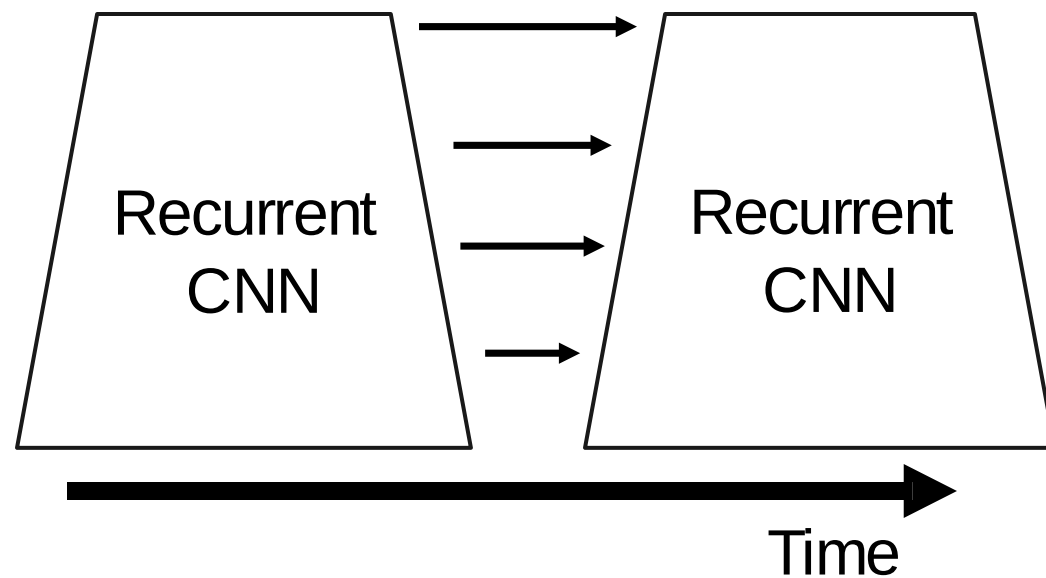
问题: RNN对于长序列很慢 (无法并行化)

RNN: Infinite
temporal extent
(fully-connected)

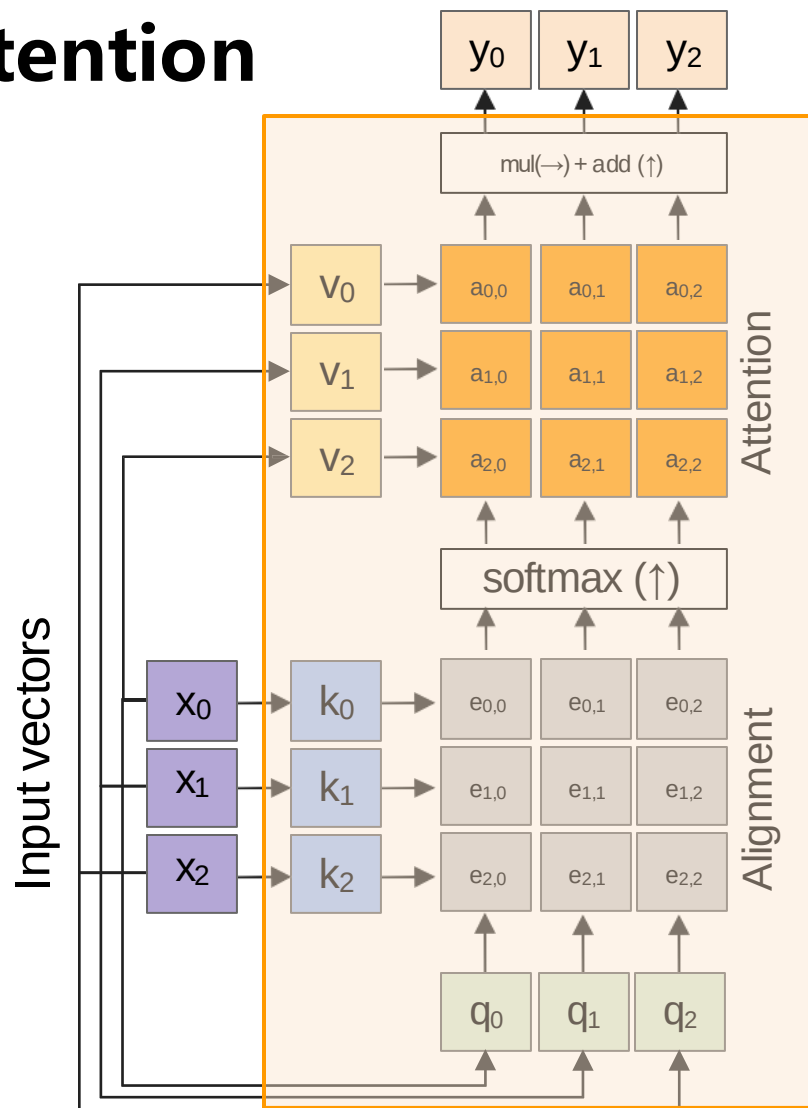
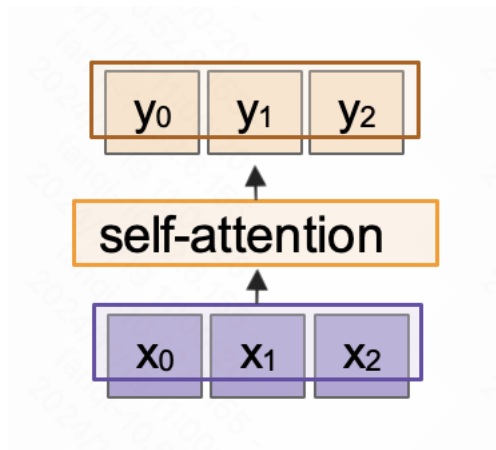
CNN: finite
temporal extent
(convolutional)



Recurrent CNN: Infinite
temporal extent
(convolutional)



■ 回顾：Self-Attention



Outputs:
context vectors: y (shape: D_v)

Operations:

Key vectors: $k = xW_k$

Value vectors: $v = xW_v$

Query vectors: $q = xW_q$

Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$

Attention: $a = \text{softmax}(e)$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

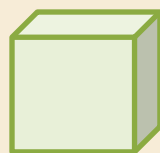
Input vectors: x (shape: $N \times D$)

■ Spatio-Temporal Self-Attention (Nonlocal Block)

Input clip



3D
CNN



Features:
 $C \times T \times H \times W$

Nonlocal Block

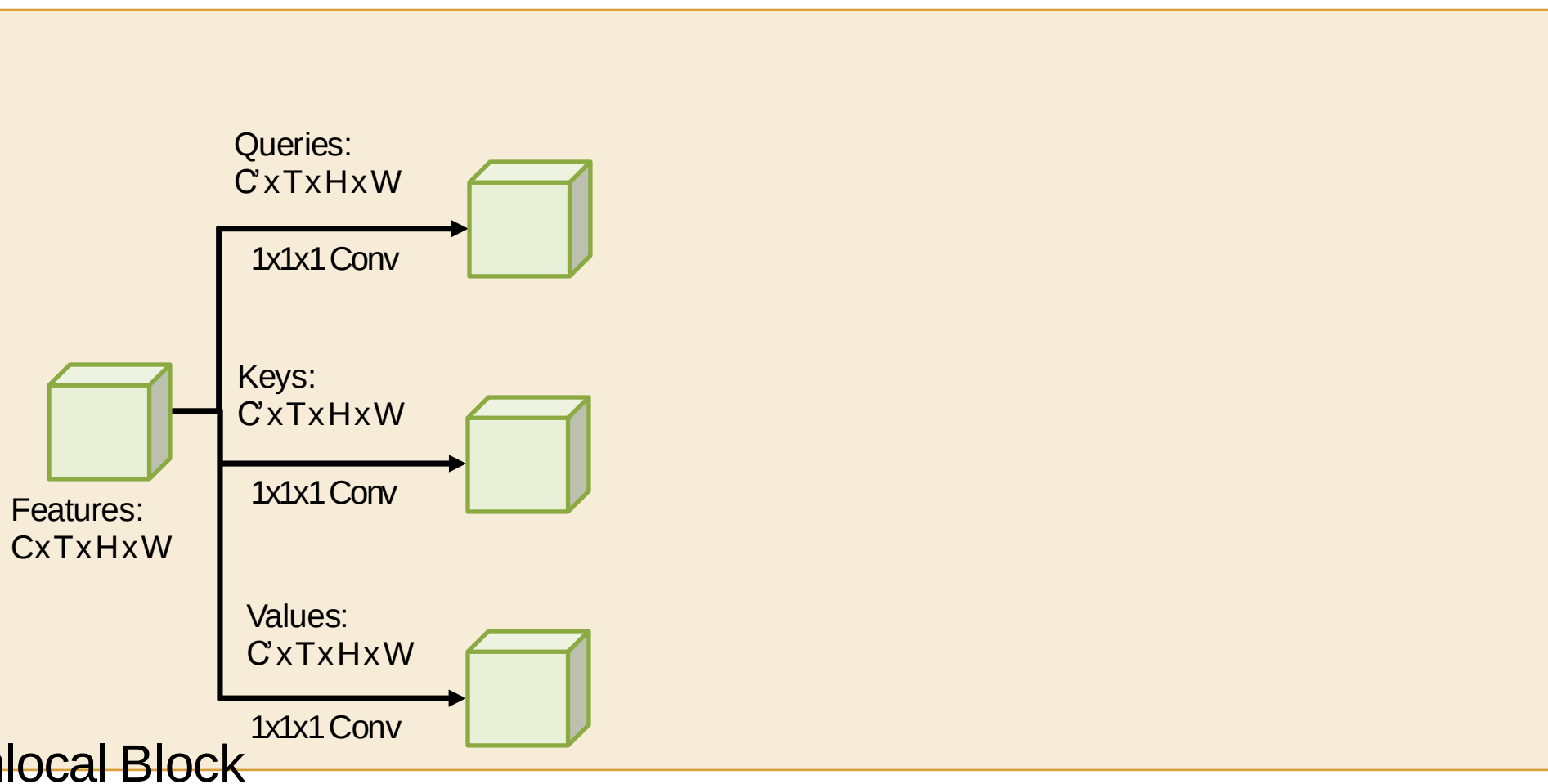
■ Spatio-Temporal Self-Attention (Nonlocal Block)

Input clip



3D
CNN

Nonlocal Block



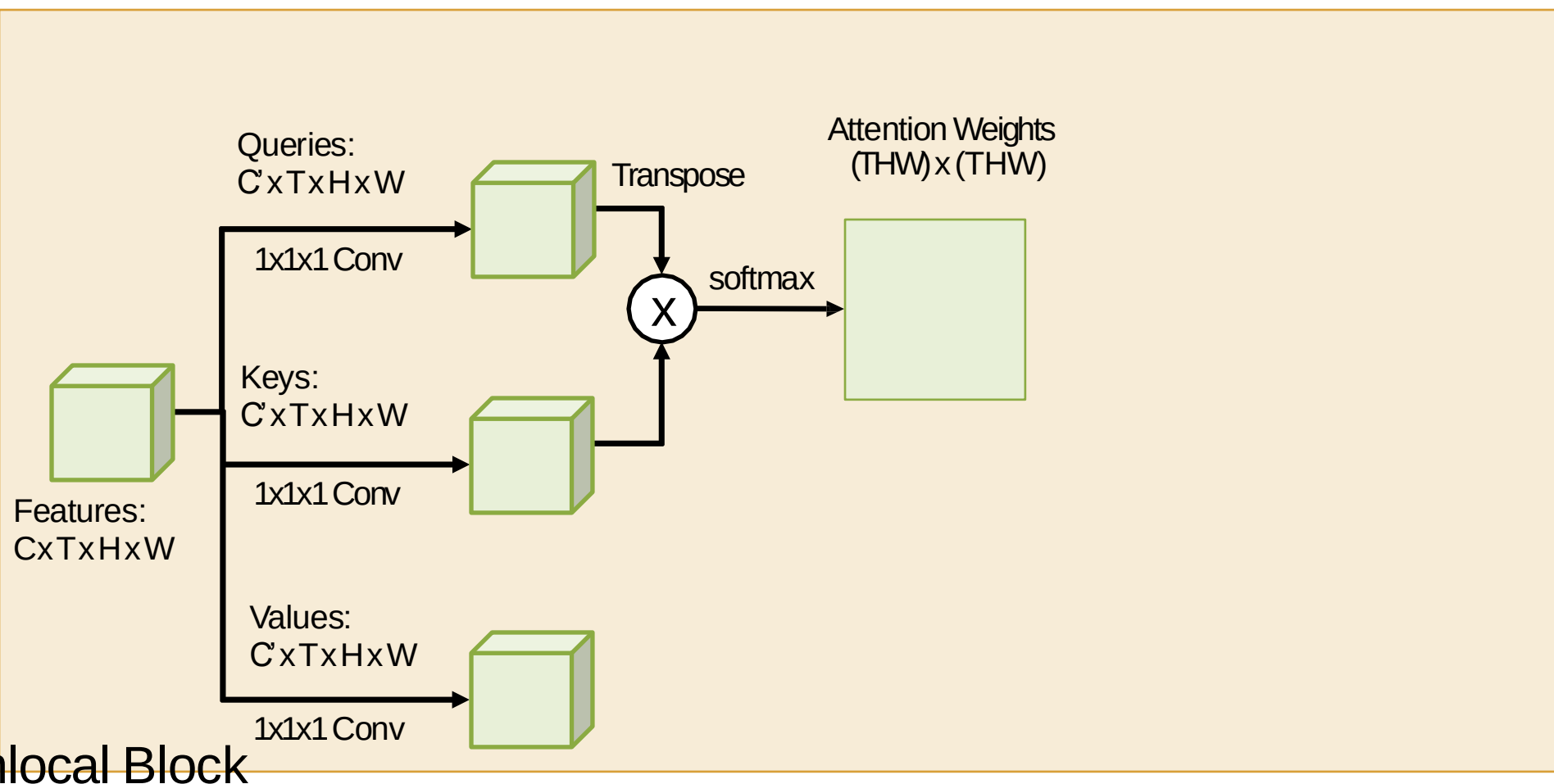
■ Spatio-Temporal Self-Attention (Nonlocal Block)

Input clip



3D
CNN

Nonlocal Block



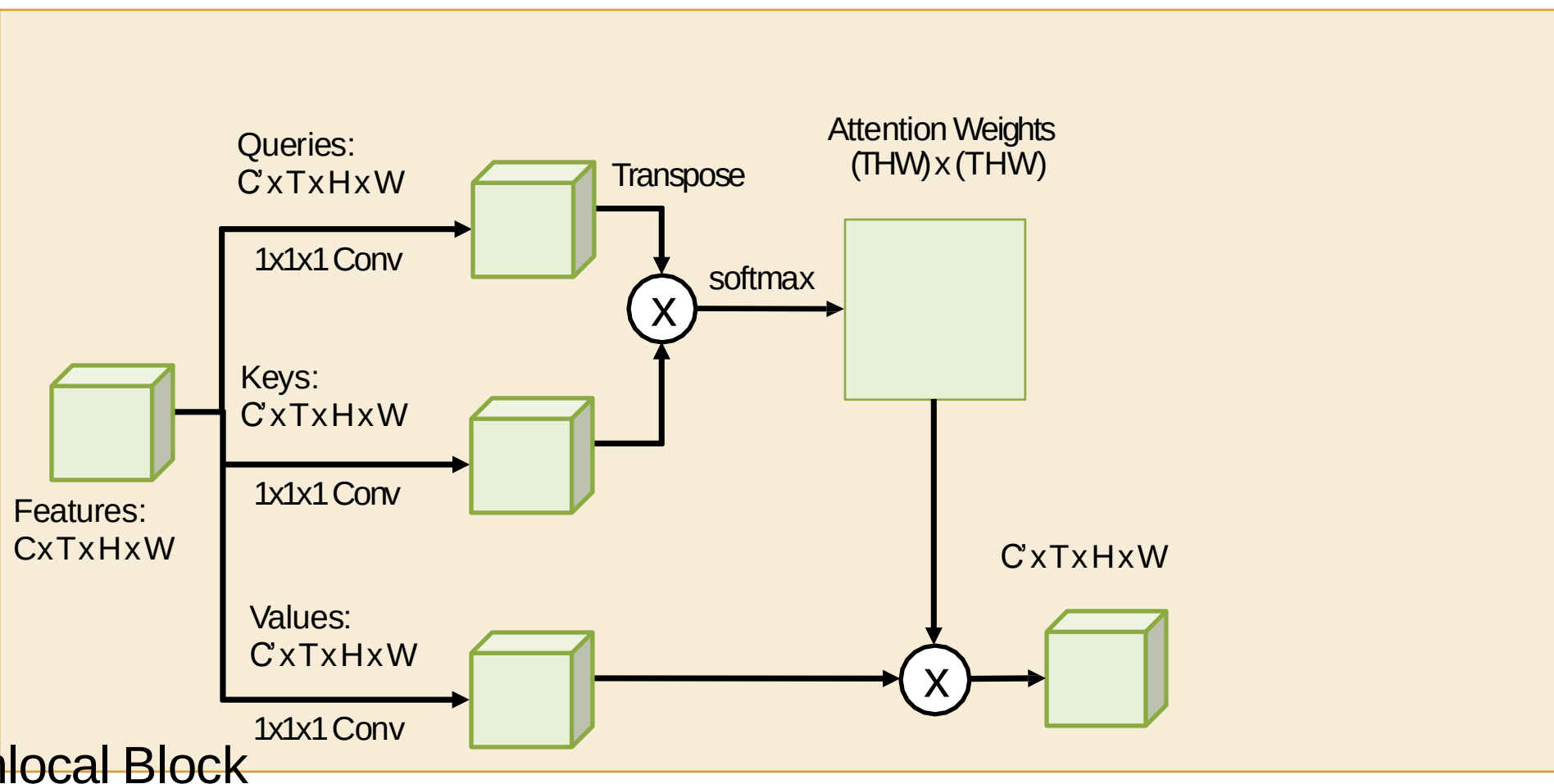
■ Spatio-Temporal Self-Attention (Nonlocal Block)

Input clip



3D
CNN

Nonlocal Block



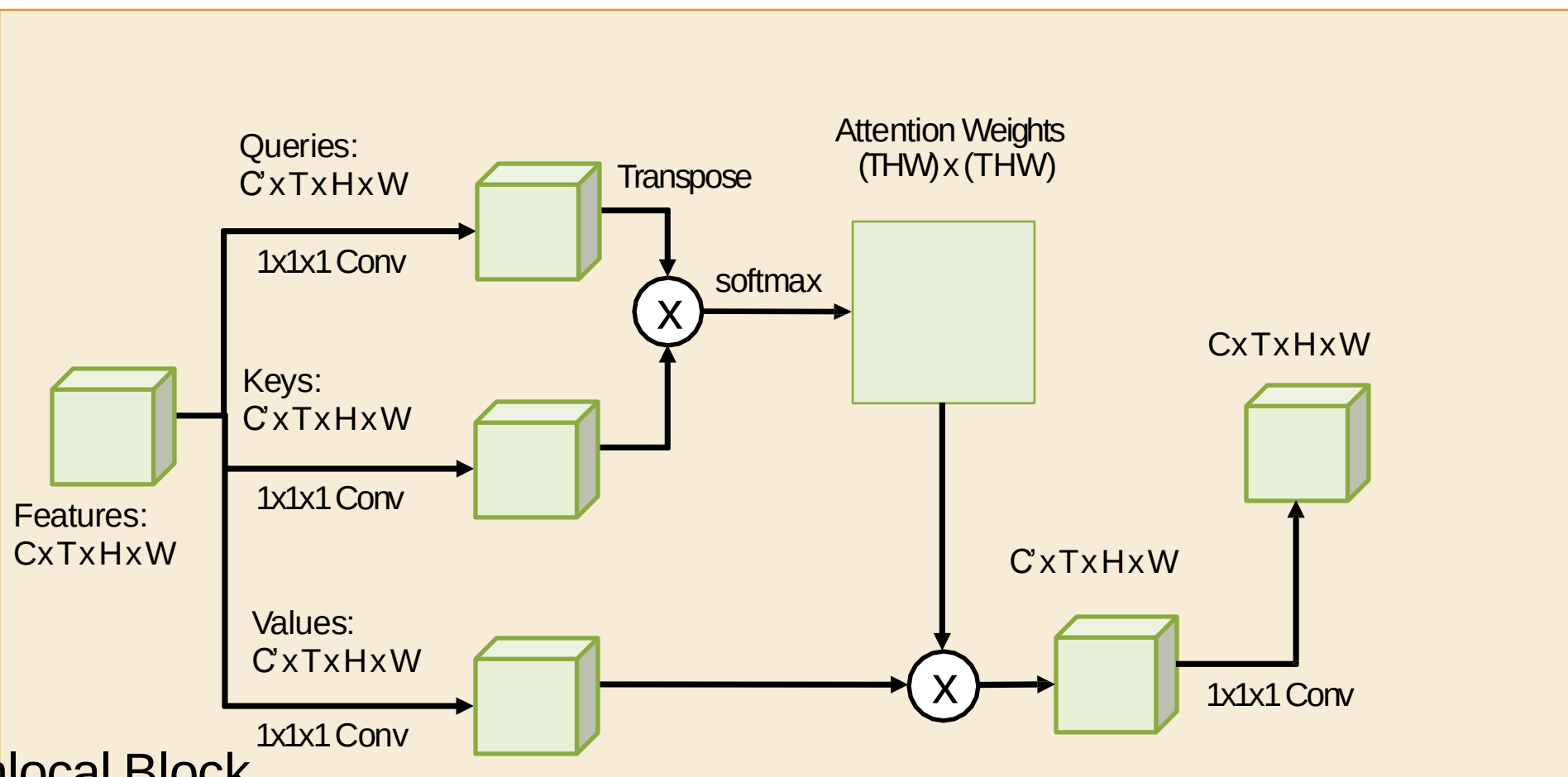
■ Spatio-Temporal Self-Attention (Nonlocal Block)

Input clip

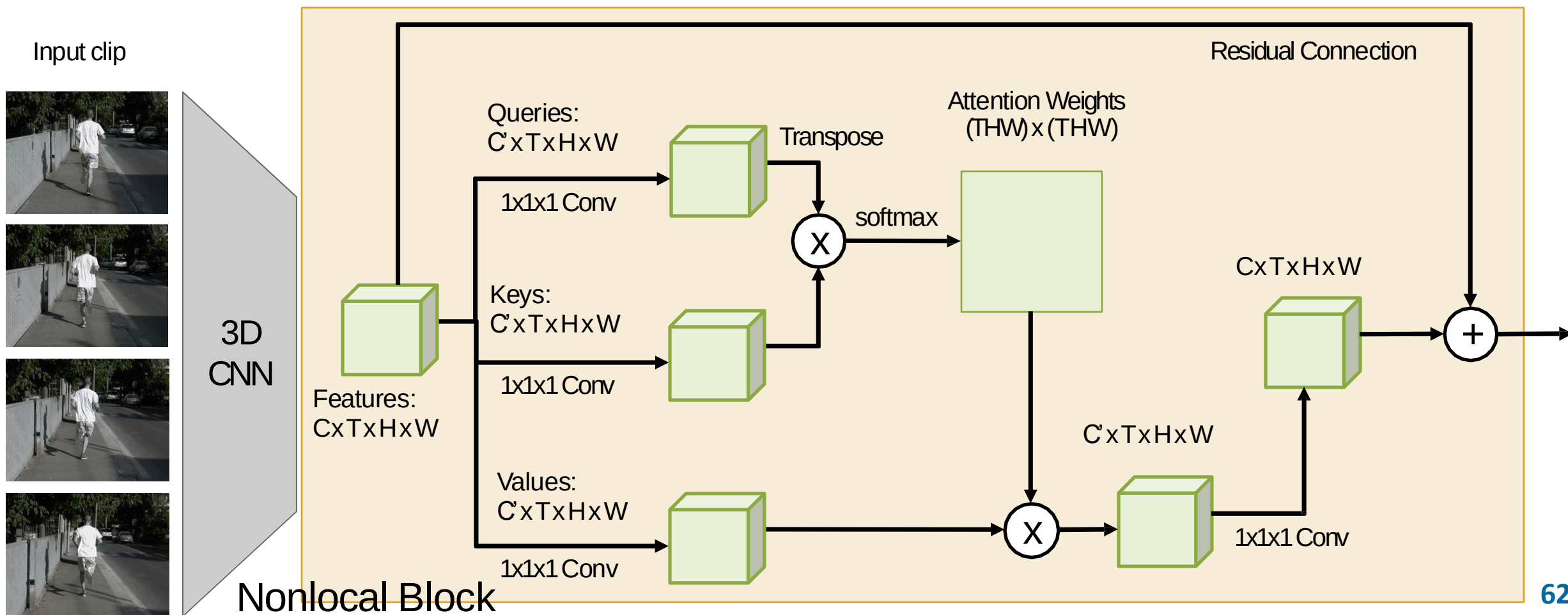


3D
CNN

Nonlocal Block



■ Spatio-Temporal Self-Attention (Nonlocal Block)

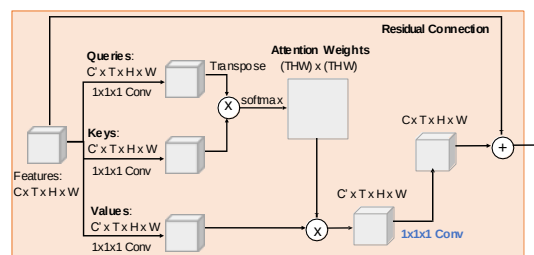


■ Spatio-Temporal Self-Attention (Nonlocal Block)

Input clip

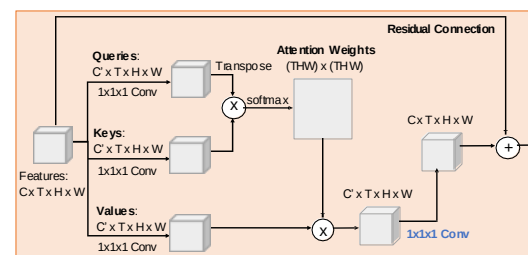


3D CNN



Nonlocal Block

3D CNN



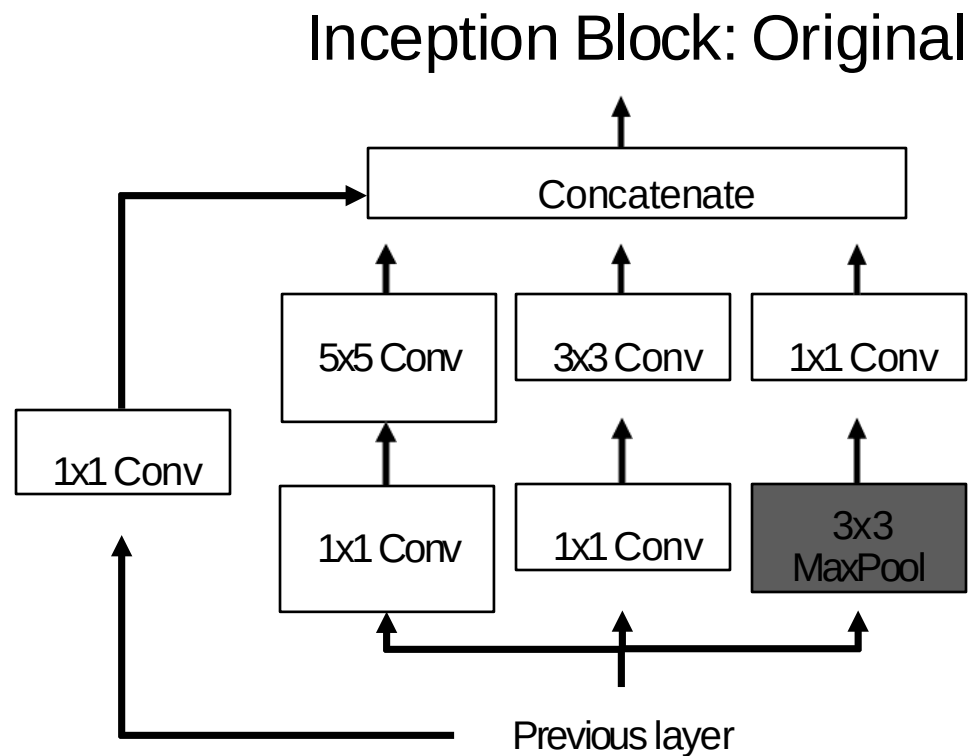
Nonlocal Block

3D CNN

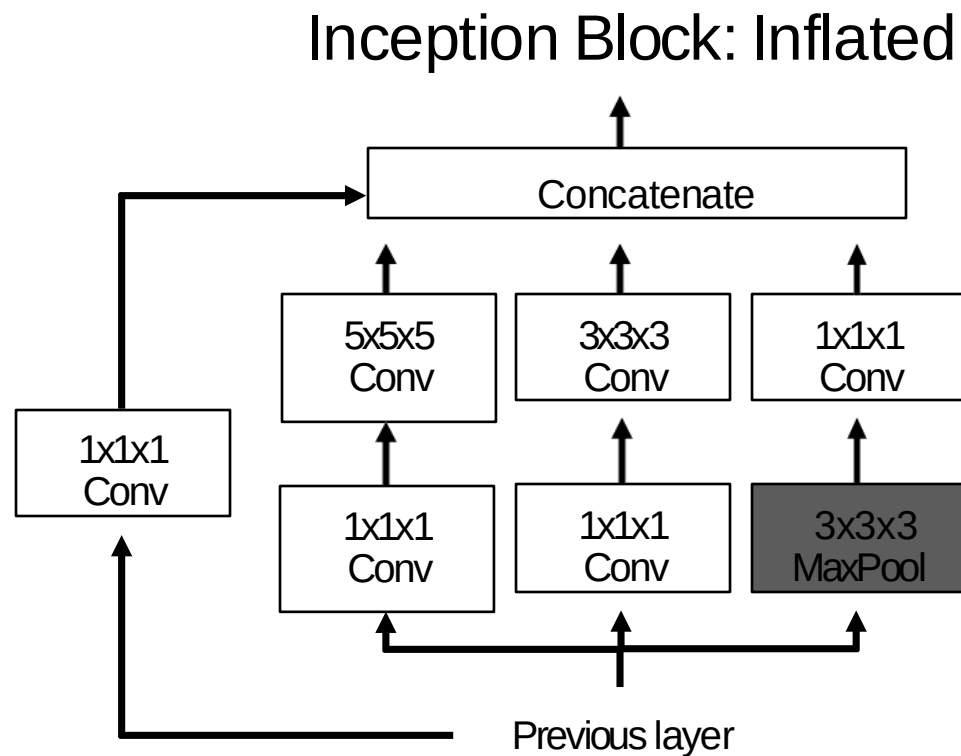
Running

- 重用 2D CNN: Inflating 2D Networks to 3D (I3D)
- 将每个 2D kernel 替换为 3D kernel

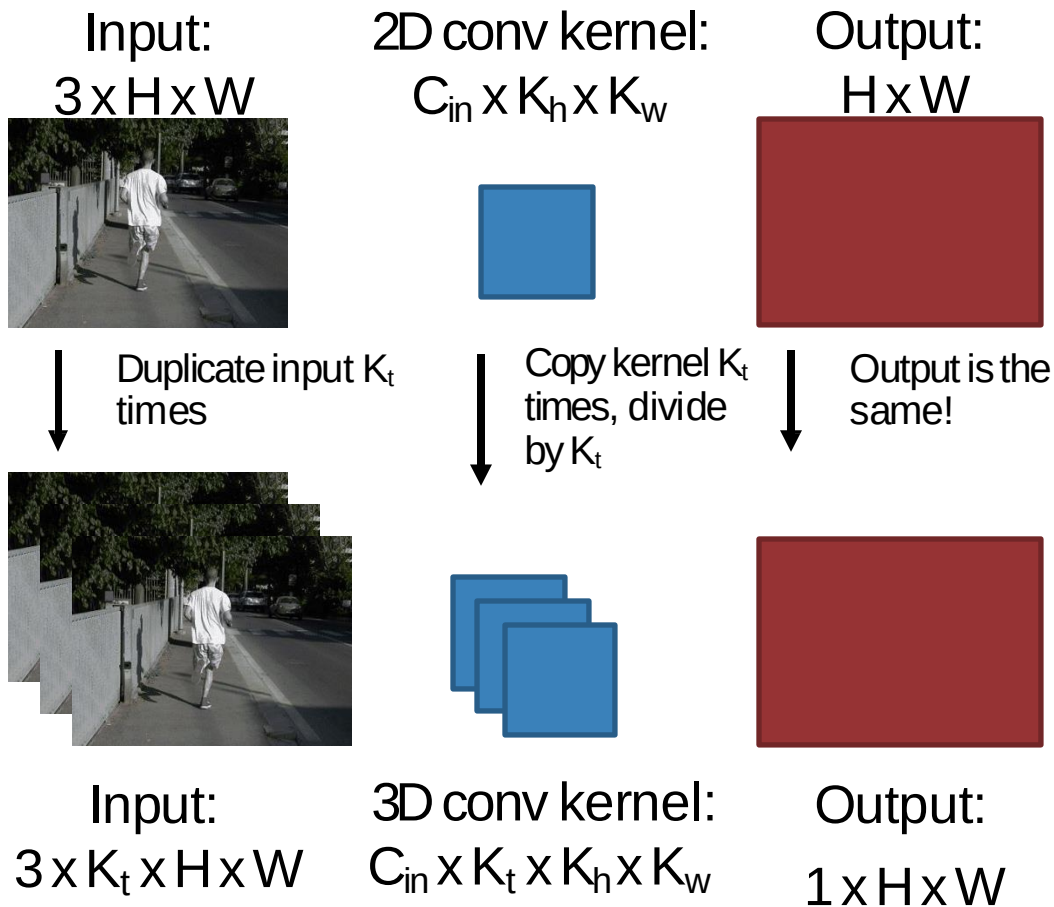
- 重用 2D CNN: Inflating 2D Networks to 3D (I3D)
- 将每个 2D kernel 替换为 3D kernel



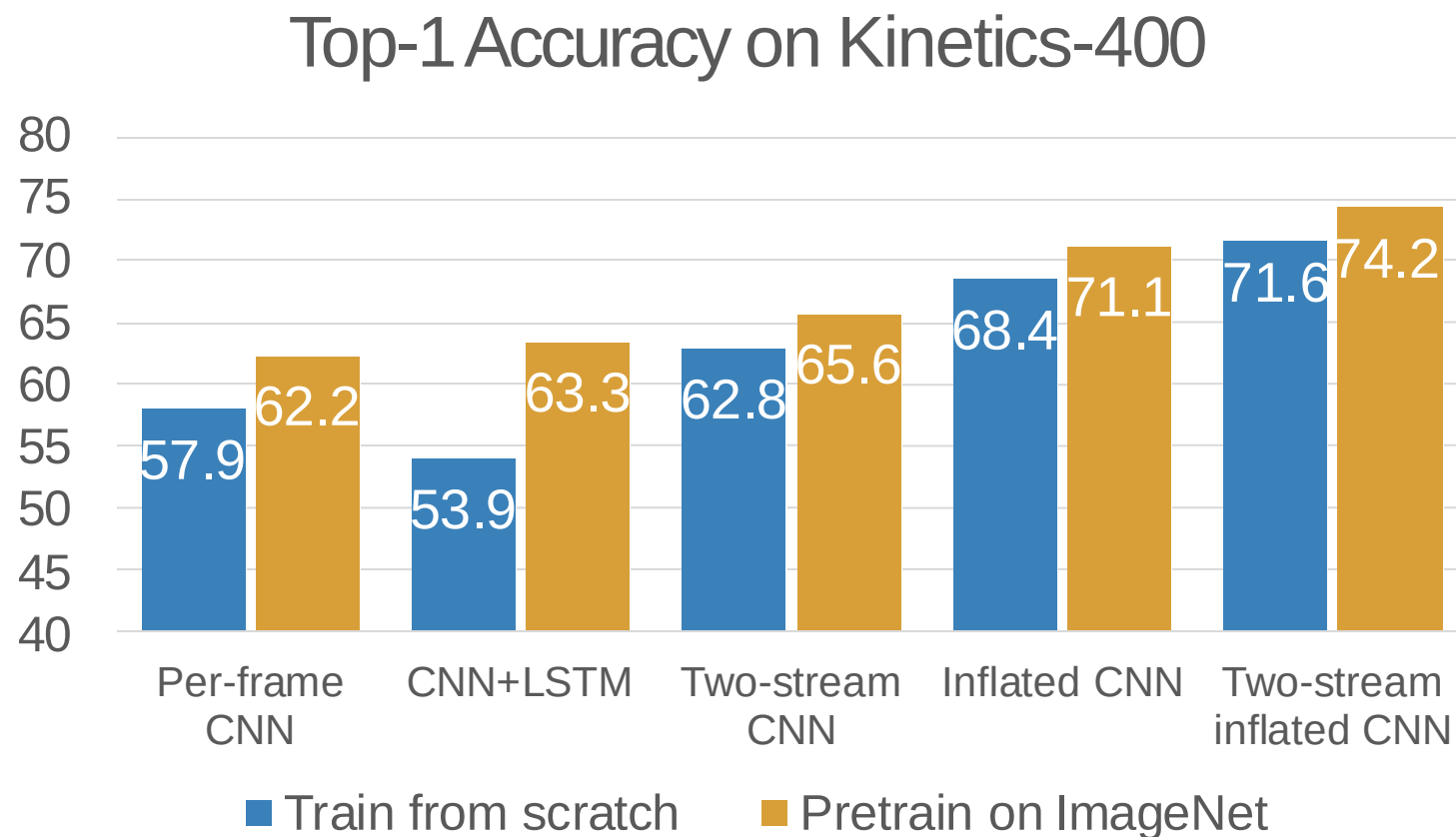
- 重用 2D CNN: Inflating 2D Networks to 3D (I3D)
- 将每个 2D kernel 替换为 3D kernel



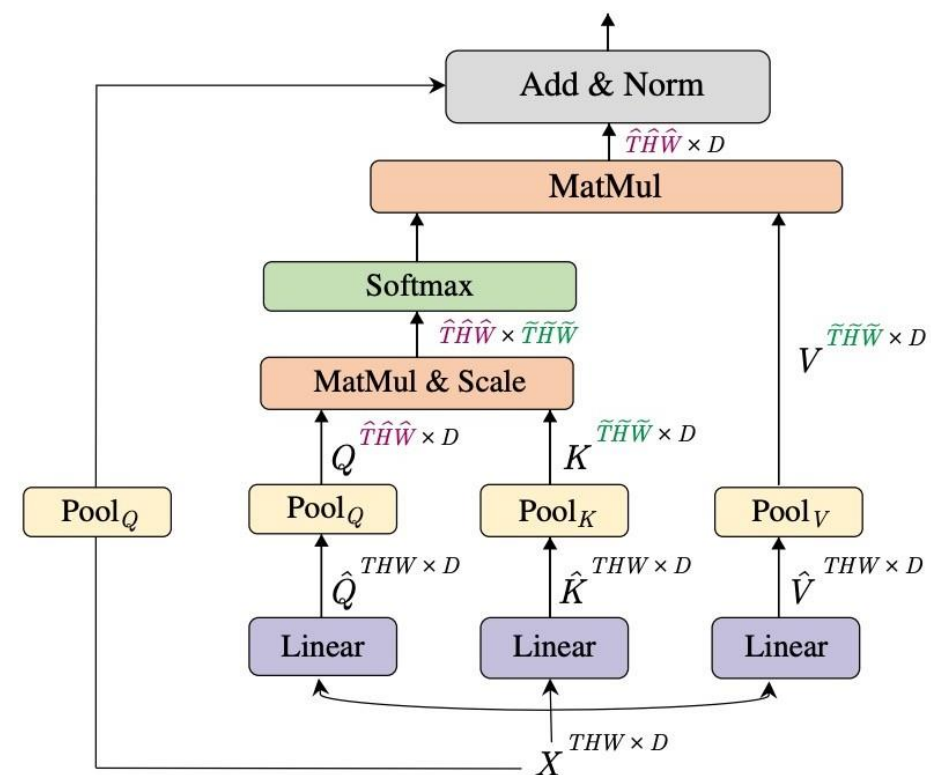
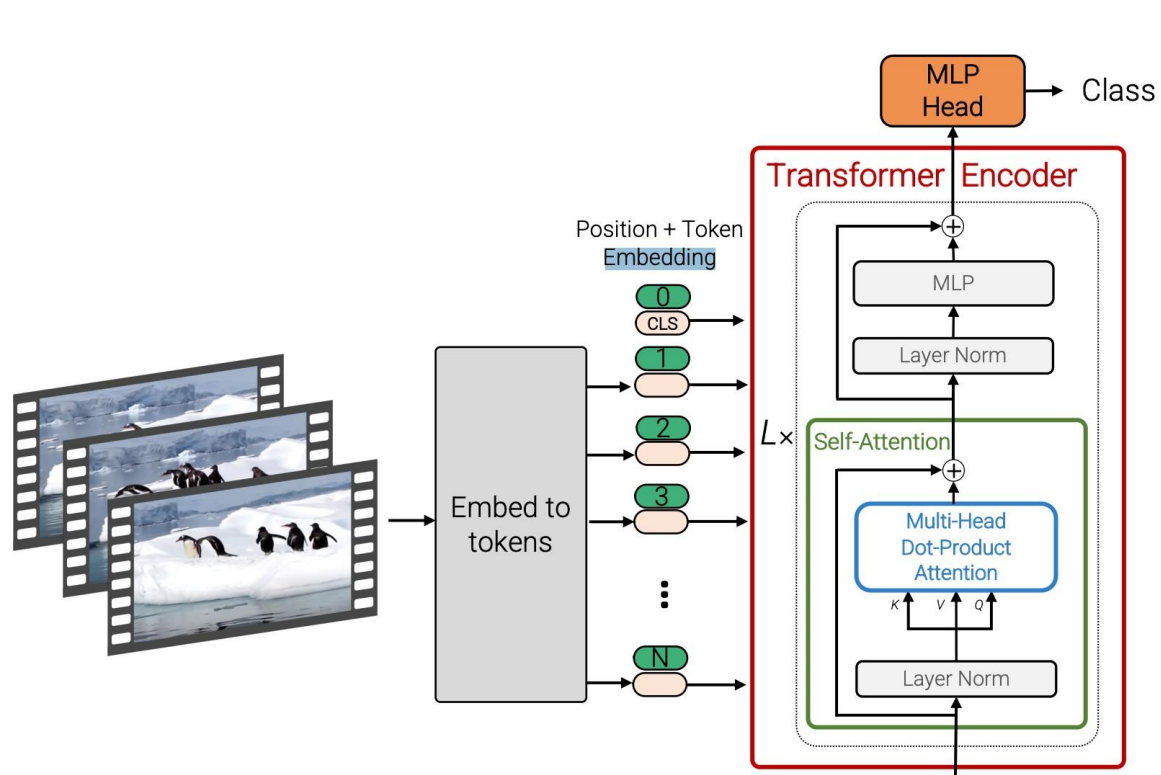
- Inflating 2D Networks to 3D (I3D)
- 可以使用 2D conv 的权重来初始化 3D conv: 在空间中复制 K_t 次并除以 K_t
- 这使得两者的预测结果相同



■ 重用 2D CNN: Inflating 2D Networks to 3D (I3D)

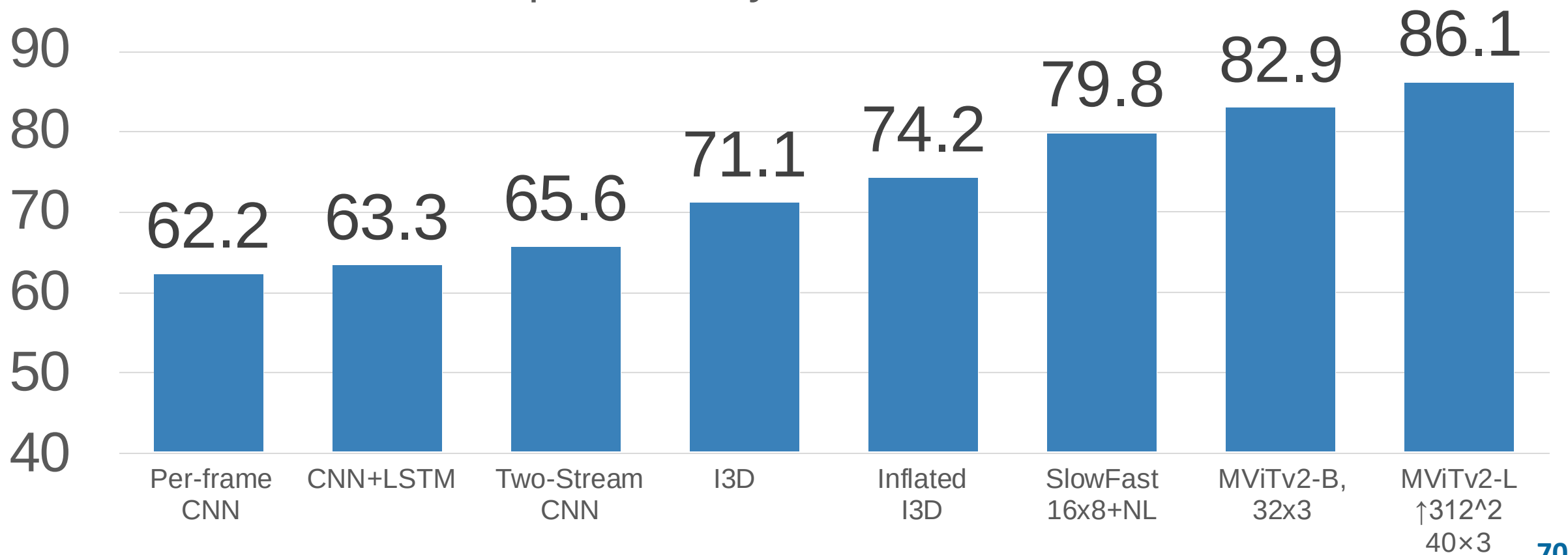


Video Transformer



■ Video Transformer

Top-1 Accuracy on Kinetics-400



■ 视频任务

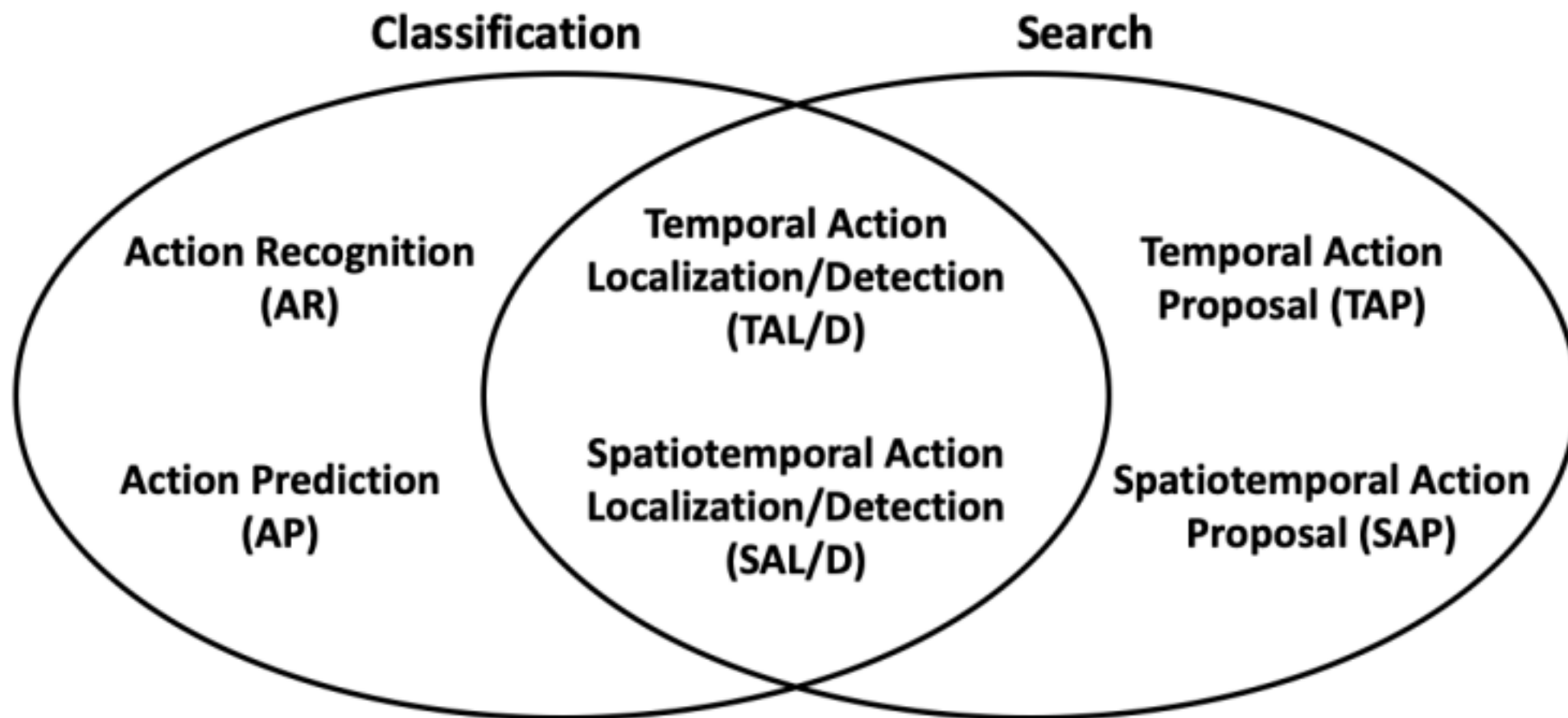


视频分类：识别动作

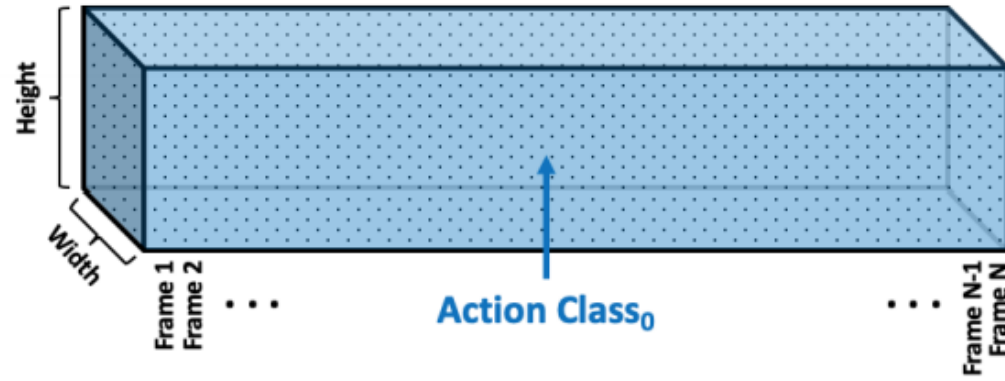


Swimming
Running
Jumping
Eating
Standing

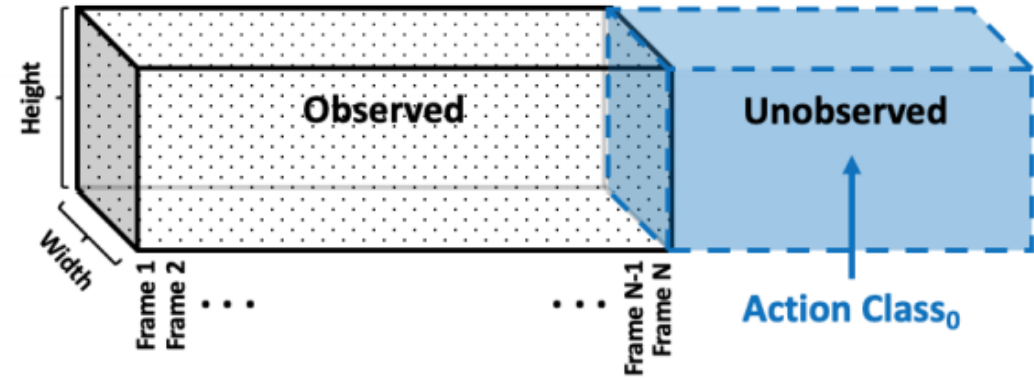
■ 视频任务



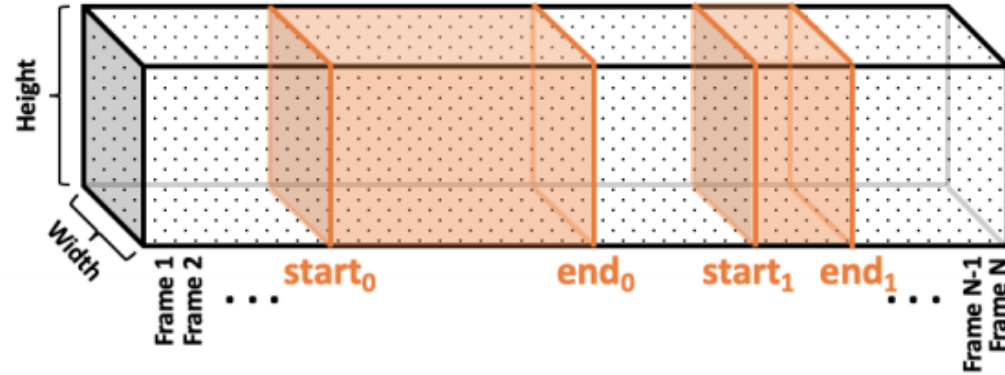
Action Recognition (AR)



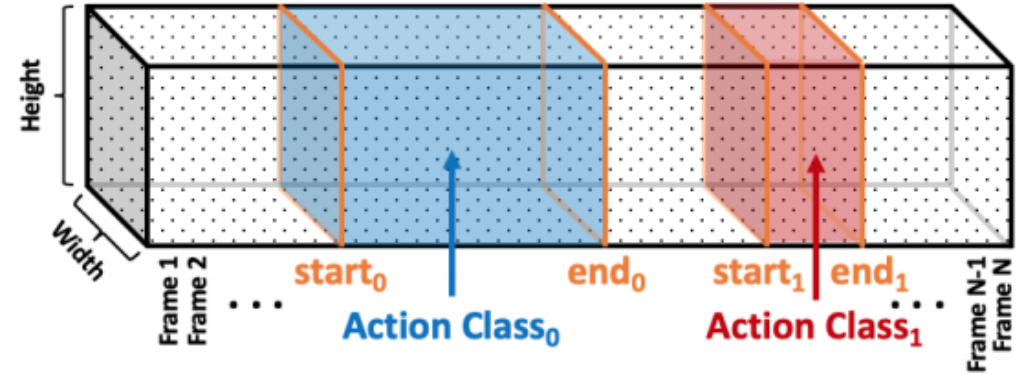
Action Prediction (AP)



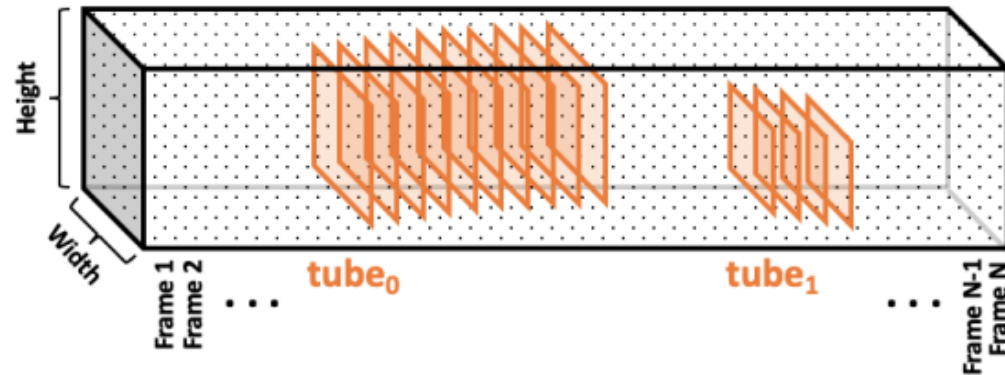
Temporal Action Proposal (TAP)



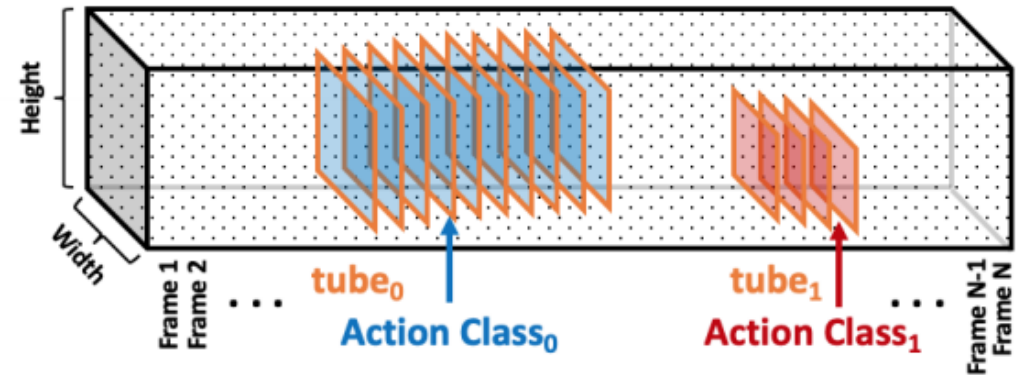
Temporal Action Localization/Detection (TAL/D)



Spatiotemporal Action Proposal (SAP)



Spatiotemporal Action Localization/Detection (SAL/D)

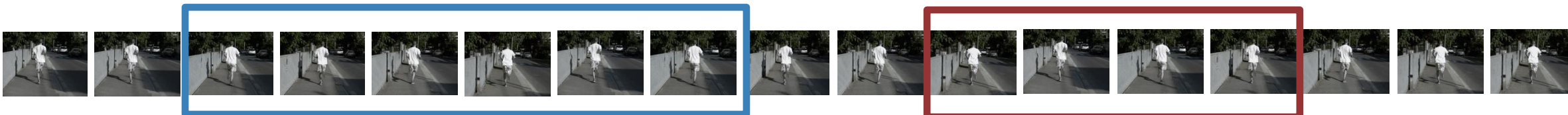


- Temporal Action Localization
- 给定一个长视频序列，识别与不同动作对应的帧
- 可以使用类似 Faster R-CNN 的架构：先生成 temporal proposals，然后进行分类

Running



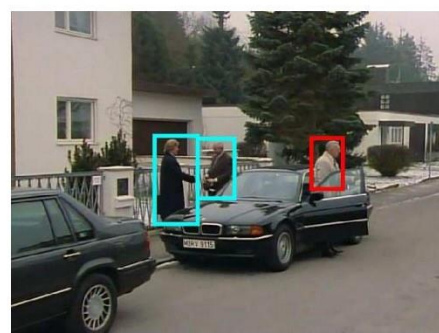
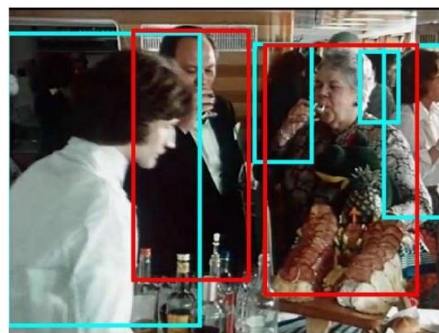
Jumping



- Spatio-Temporal Detection
- 给定一个长视频序列，检测空间和时间中的所有人，并对他们正在进行的活动进行分类



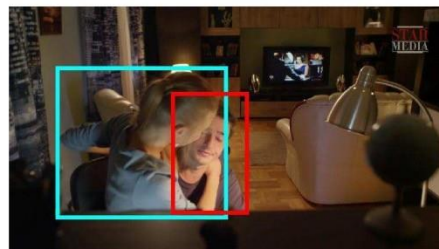
clink glass → drink



open → close



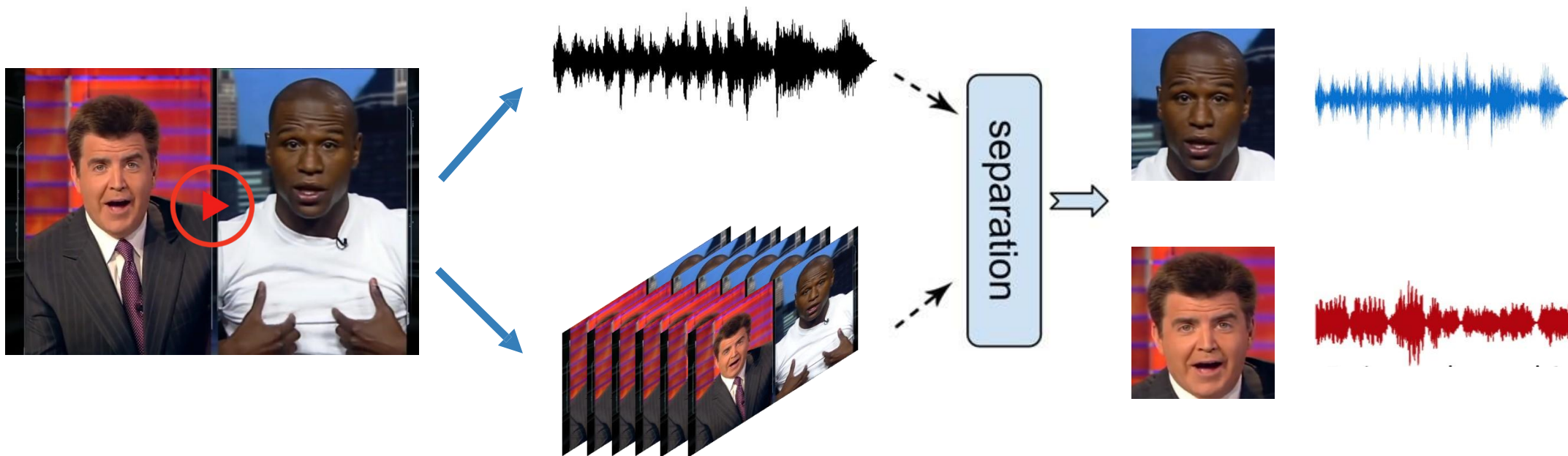
grab (a person) → hug



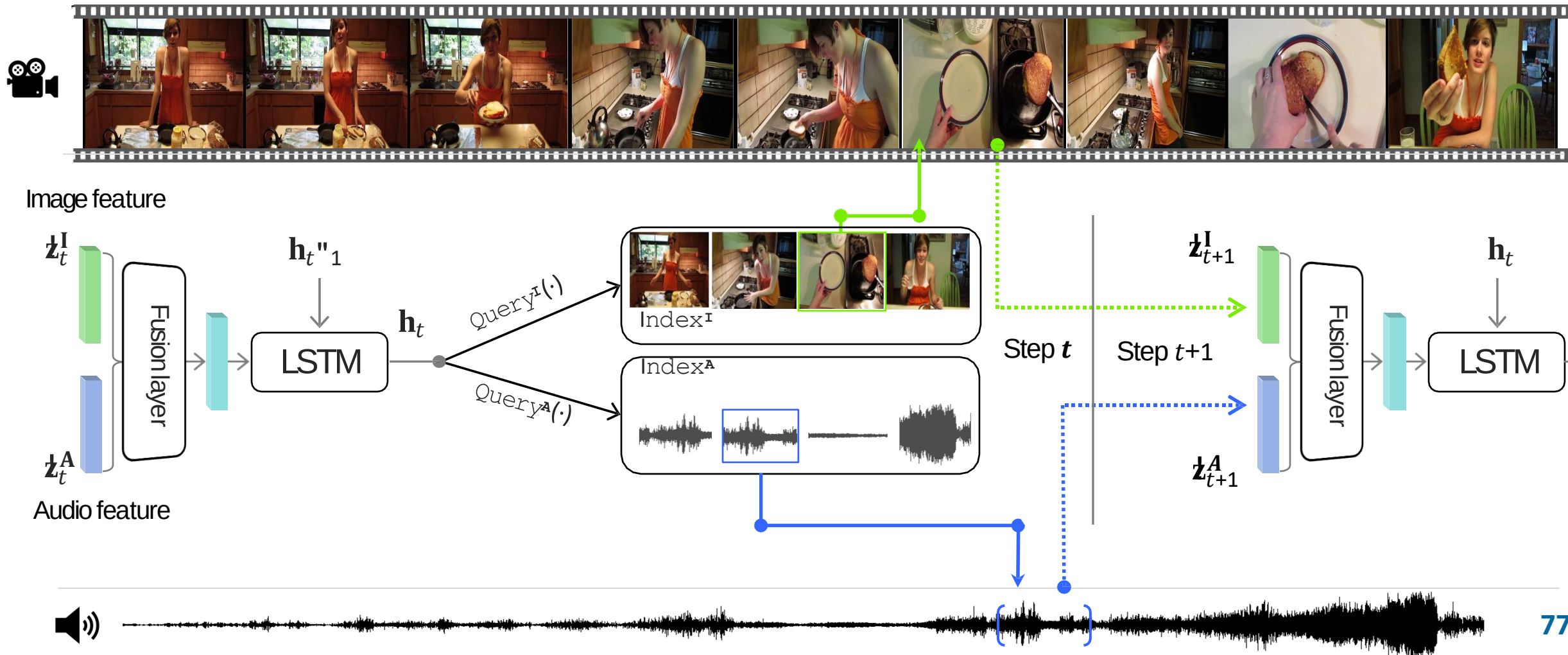
look at phone → answer phone



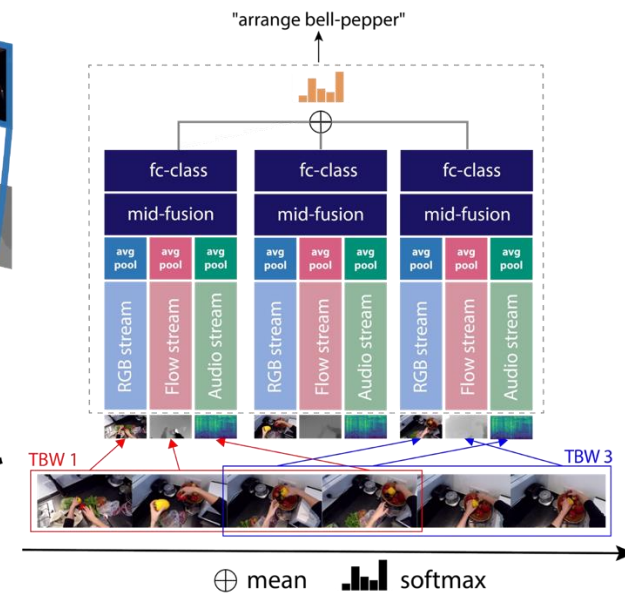
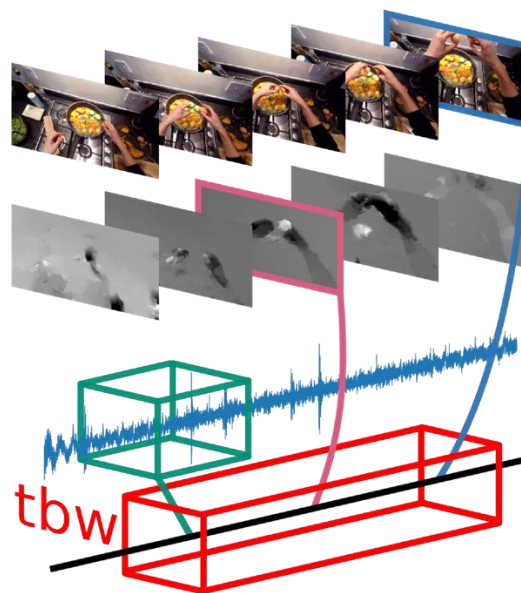
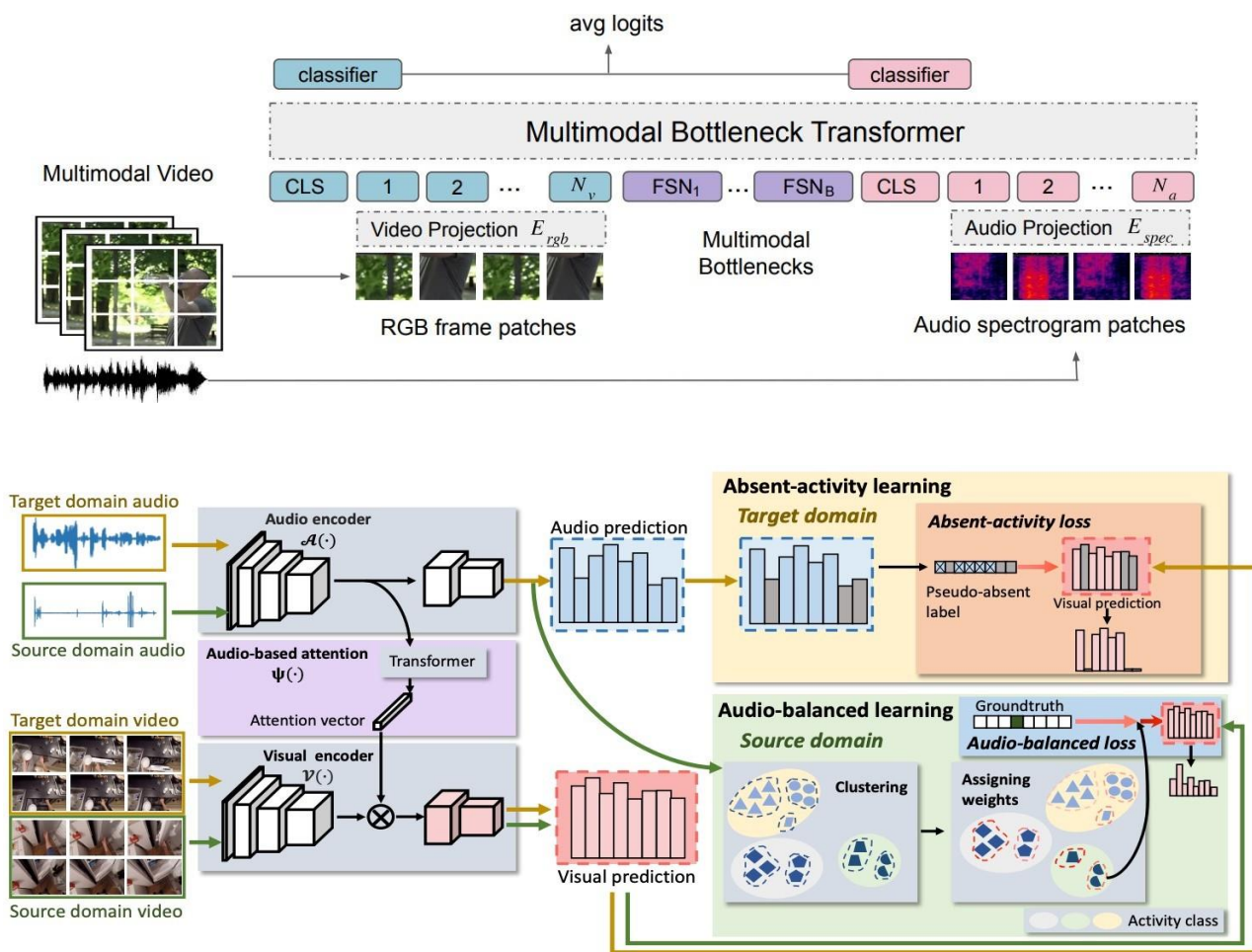
■ Visually-guided audio source separation



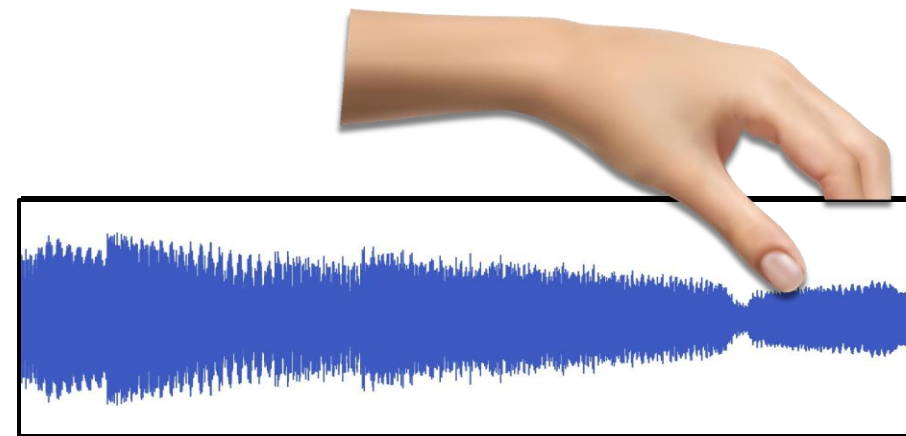
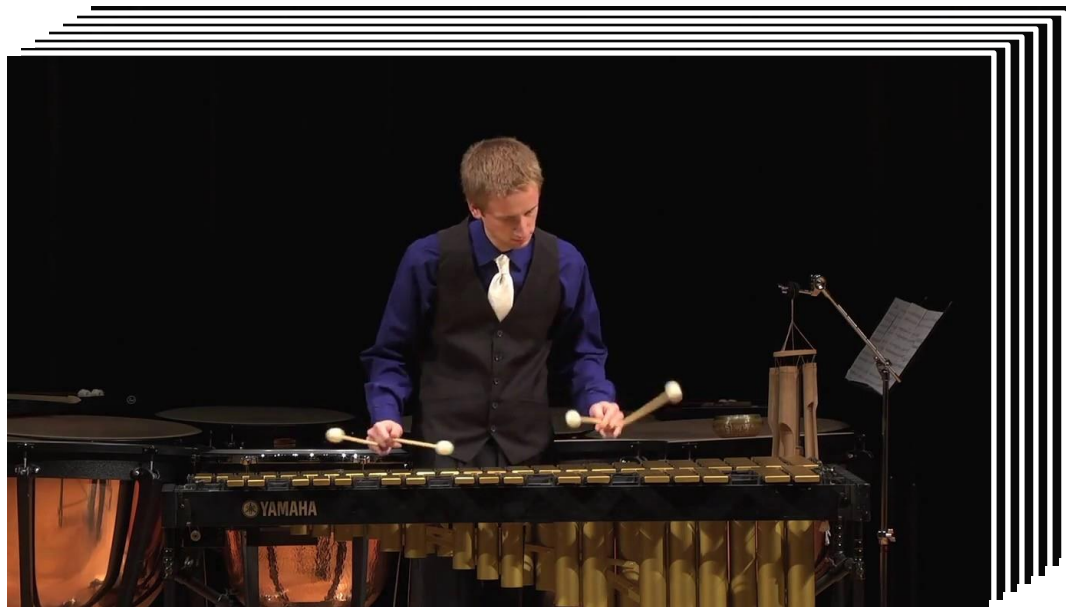
■ 利用音频进行高效率的视频动作识别



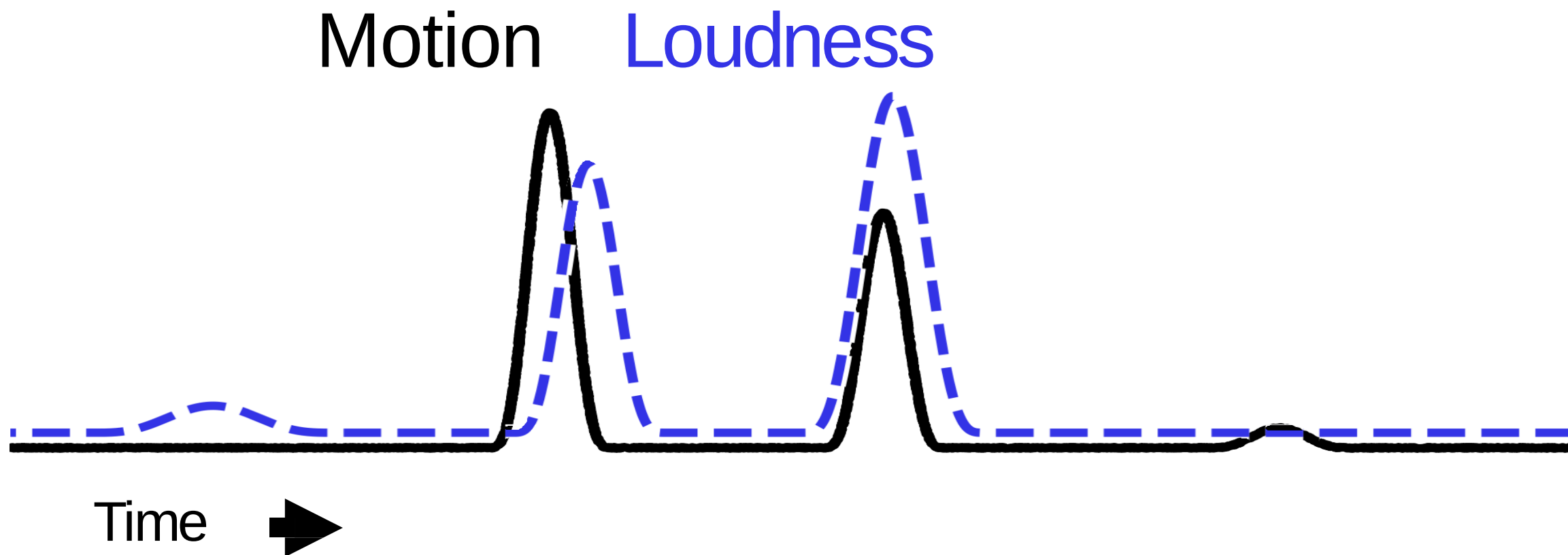
多模态视频理解



■ 视听同步



■ 视听同步



■ 视听同步

Aligned vs. misaligned

